

ЛЕКЦИЯ 1

ВВЕДЕНИЕ В ВЕБ-РАЗРАБОТКУ
GIT / GITHUB / GITHUB PAGES
НАСТРОЙКА IDE

ВОПРОСЫ

1. Структура курса
2. Введение в веб-разработку
3. GIT / GitHub / GitHub Pages
4. IDE (VS code) + настройки + установка

1

СТРУКТУРА КУРСА

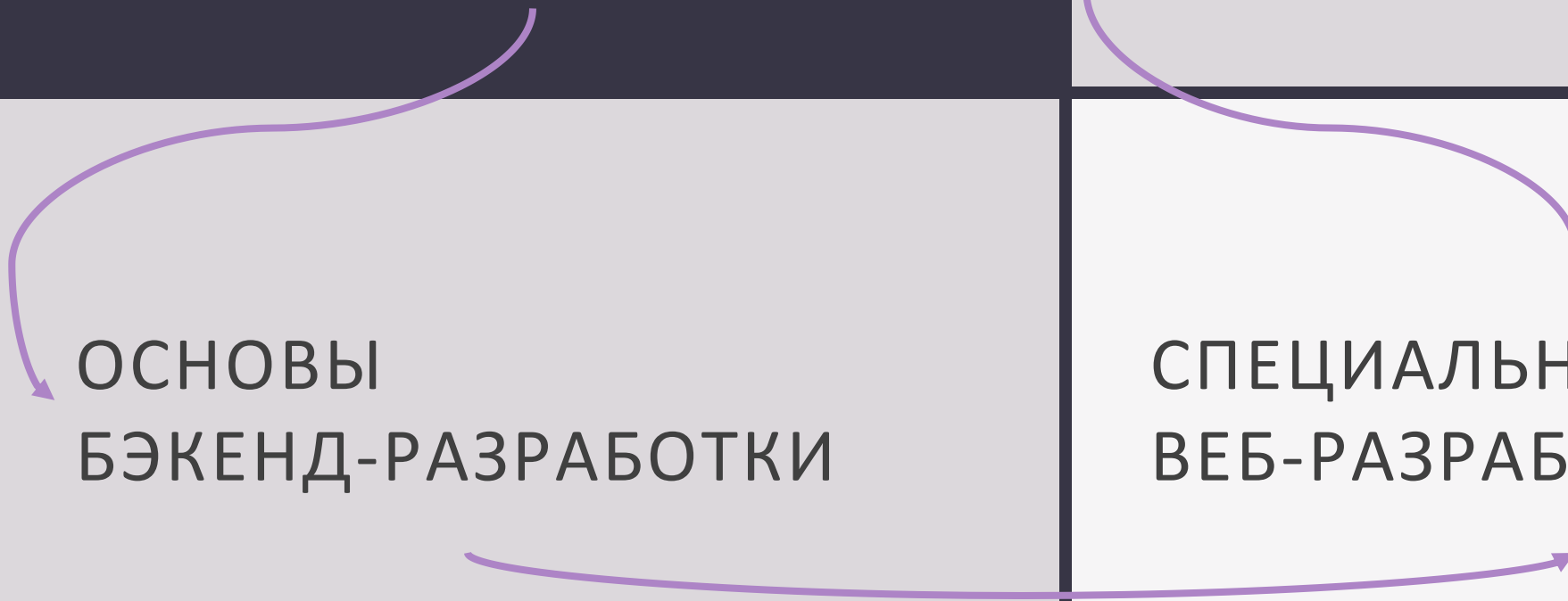
СТРУКТУРА КУРСА

ОСНОВЫ
ФРОНТЕНД-РАЗРАБОТКИ

ПРОЕКТ = ИТОГ КУРСА

ОСНОВЫ
БЭКЕНД-РАЗРАБОТКИ

СПЕЦИАЛЬНЫЕ ГЛАВЫ
ВЕБ-РАЗРАБОТКИ



ОСНОВЫ ФРОНТЕНД-РАЗРАБОТКИ

GIT, GITHUB, GITHUB PAGES

HTML

CSS

АДАПТИВНАЯ ВЕРСТКА

ФРЕЙМВОРКИ. ЧАСТЬ 1

МОБИЛЬНАЯ РАЗРАБОТКА

АНАЛИЗ ЦА И КОНКУРЕНТОВ

JAVASCRIPT

ФРЕЙМВОРКИ. ЧАСТЬ 2

ФРЕЙМВОРКИ. ЧАСТЬ 3

2

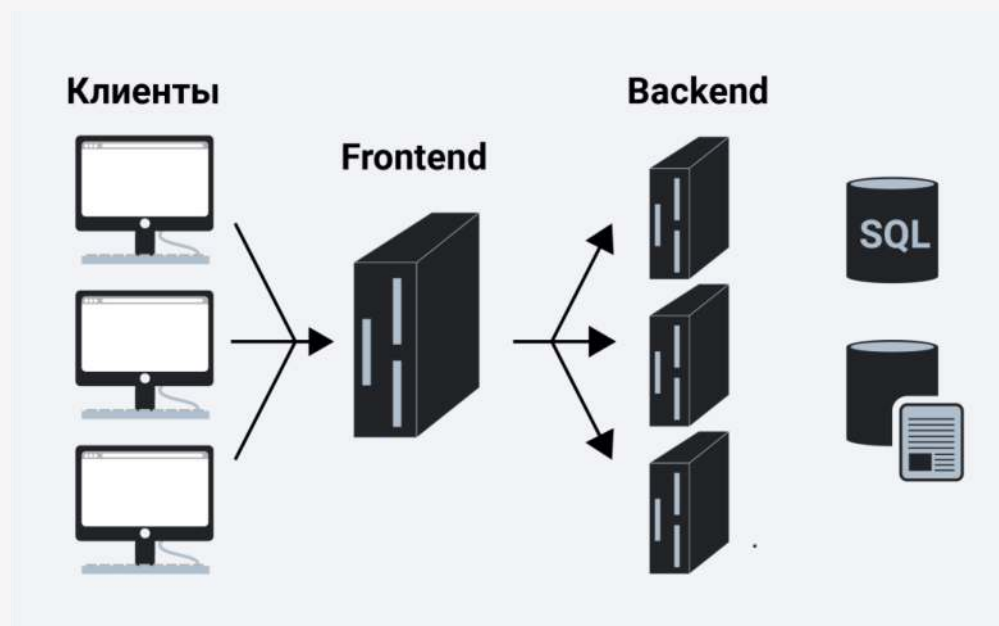
ВВЕДЕНИЕ В ВЕБ-РАЗРАБОТКУ

Веб-разработка



Веб-разработка — это создание клиент-серверных программных систем, доступ к которым осуществляется через браузер или иные HTTP-клиенты.

Типовая структура веб-приложения включает клиентскую часть (frontend), серверную часть (backend), интерфейсы взаимодействия (API), подсистемы хранения данных, а также средства доставки статического контента и автоматизации сборки и развёртывания



Компоненты типового веб-приложения

Перед тем как перейти к классификации и архитектуре, выделим базовые элементы и их связи: это поможет соотнести термины курса с реальными артефактами:

- **Клиент (Frontend).** Браузер, который интерпретирует HTML/CSS/JS, управляет рендерингом интерфейса, выполняет логику на стороне клиента и обращается к API.
- **Сервер (Backend).** Приложение, реализующее бизнес-правила, авторизацию, валидацию и доступ к данным; предоставляет API (обычно REST/JSON или GraphQL).
- **СУБД и хранилища.** Реляционные (PostgreSQL, MySQL) и документо-ориентированные (MongoDB) базы; кэш (Redis), очереди (RabbitMQ).
- **Доставка статического контента.** CDN/Pages для HTML/CSS/JS/изображений; сокращает задержки и разгружает сервер.
- **CI/CD.** Средства сборки, тестирования и развертывания; автоматизируют публикацию изменений и контроль качества

Классификация веб-приложений

Классификация веб-приложений помогает соотнести требования (функциональные и нефункциональные) с подходящими архитектурными и технологическими решениями



1. По типу представляемого контента

1. По типу представляемого контента

1.1 Статические сайты

- Контент подготовлен заранее (HTML/CSS/JS) и выдаётся без серверной генерации на каждый запрос.
- Чистый HTML/CSS/JS; статические генераторы (по необходимости).
- Личные страницы, портфолио, лендинги, документация.
- Простая публикация (в т.ч. GitHub Pages), высокая скорость, низкая стоимость.
- Ограниченная интерактивность без внешних API; сложнее реализовать персонализацию на стороне сервера.

1.2 Динамические сайты

- Контент формируется «на лету» с учётом параметров запроса и состояния системы.
- Серверные языки/фреймворки, БД (SQL/NoSQL), кеши.
- Интернет-магазины, социальные сети, системы бронирования.
- Сильные стороны. Персонализация, сложная бизнес-логика, работа с данными.
- Ограничения. Более высокая сложность разработки и эксплуатации, требования к безопасности и масштабированию.

Статические сайты. Содержимое формируется заранее (HTML/CSS/JS) и отдаётся без серверной генерации на каждый запрос. Типичные примеры: персональные страницы, портфолио, лендинги, документация.

Динамические сайты. Содержимое формируется при обработке запроса с учётом параметров пользователя и состояния системы; используется серверный код и база данных. Примеры: электронная коммерция, социальные сети.

2. По организации пользовательского интерфейса

2. По организации пользовательского интерфейса

2.1 Многостраничные приложения (MPA)

Признаки. Каждый значимый переход — загрузка новой страницы с сервера.

Когда применять. Крупные контентные порталы, корпоративные системы, SEO-чувствительные ресурсы.

Особенности. Простая модель, хорошая индексация; больше полных перезагрузок.

2.2 Одностраничные приложения (SPA)

Признаки. Интерфейс загружается единожды; дальнейшие изменения — на клиенте, обмен с сервером по API.

Когда применять. Богатые интерактивные интерфейсы, сложные клиентские сценарии.

Особенности. Высокая интерактивность; важно контролировать производительность и начальный TTI.

2.3 Стратегии рендеринга

CSR (Client-Side Rendering). Рендеринг в браузере; минимальный сервер.

SSR (Server-Side Rendering). Рендеринг на сервере для ускорения первого экрана и SEO.

SSG (Static Site Generation). Генерация страниц на этапе сборки; выдача как статических файлов.

ISR (Incremental Static Regeneration). Частичное обновление статической выдачи по мере устаревания.

Многостраничные приложения (MPA).

Навигация выполняется между отдельными страницами; каждый значимый переход инициирует загрузку новой страницы.

Одностраничные приложения (SPA).

Интерфейс загружается единожды; дальнейшие изменения выполняются на клиенте, обмен с сервером ведётся через API (как правило, JSON по HTTP(S))

Передача и отрисовка контента могут быть организованы по-разному; чтобы корректно соотнести требования к первому экрану и SEO с технической реализацией

3. По целевому назначению



Целевое назначение указывает на доменную область применения и типовые пользовательские сценарии, определяя требования к функциональности и качественным атрибутам (производительность, надёжность, доступность)

4. По применяемым компонентам моделям (сервер)

Без формальной модели — небольшие приложения на скриптовых языках: проще старт, меньше «церемоний», но труднее масштабировать и расширять.

Расширения веб-серверов — поддержка модульности через плагины/модули; можно наращивать функциональность без переписывания ядра.

Специализированные стандарты — например, Java Servlet: чёткий контракт обработчика запроса/ответа, фильтры, жизненный цикл. Это повышает предсказуемость и переносимость кода между контейнерами.



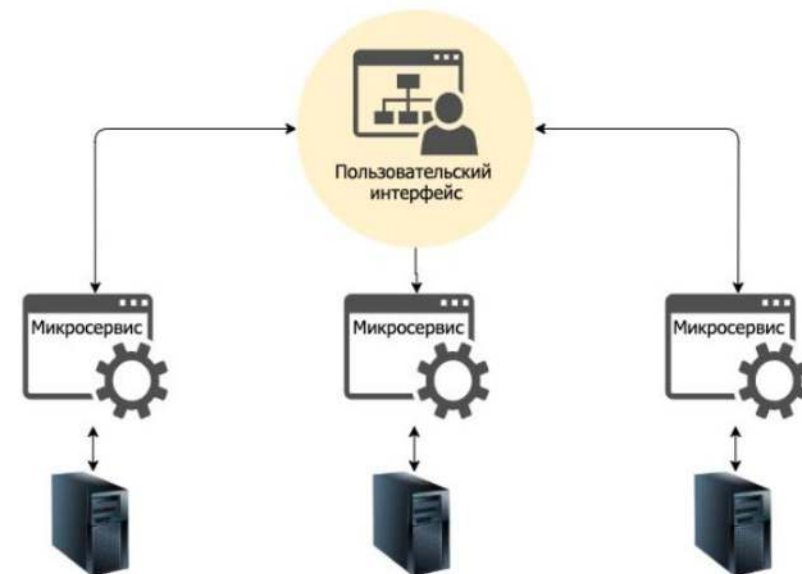
5. По архитектурному стилю

Архитектура — это состав подсистем и способы их взаимодействия для выполнения функциональных и нефункциональных требований. От неё зависят структура команд, частота релизов, наблюдаемость и сценарии масштабирования.

На практике чаще всего: монолит или микросервисы. В любом случае важны чёткие границы ответственности и стабильные контракты между частями системы.



Монолитная архитектура



Микросервисная архитектура

5. По архитектурному стилю

Монолит — всё в одном приложении.

Плюсы: быстрый старт, проще трассировка и отладка.

Минусы: со временем растёт связность, сложнее независимые релизы и масштабирование по частям.

Микросервисы — набор автономных сервисов со своими данными и контрактами.

Плюсы: независимые релизы и гибкое масштабирование.

Минусы: сложнее инфраструктура, наблюдаемость и управление контрактами, сетевые издержки.

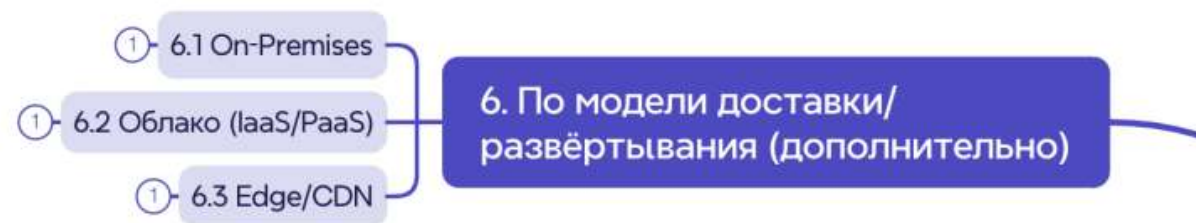


Serverless/FaaS — развёртываем функции, масштабирование берёт на себя платформа.

Плюсы: быстрое включение и эластичность.
Минусы: холодные старты, управление состоянием и профилирование.

6. По модели доставки/развертывания

От выбранной модели зависят стоимость, скорость релизов, задержки и безопасность. Смотрим: где крутится код, кто его обслуживает и как масштабируемся



On-Premises — свои серверы и сети, полный контроль и соответствие регуляторике; дорого в обслуживании, медленно масштабируется.

Облако (IaaS/PaaS) — управляемые ресурсы и сервисы: быстрый старт и эластичность, меньше рутины, но риски привязки к провайдеру и вопросы юрисдикции данных.

Edge/CDN — раздача статики/кэшируемых ответов «ближе к пользователю»: ниже задержки и нагрузка на бэкенд; важно правильно инвалидировать кэш и разделять публичные/персональные данные.

Проектирование
Создаем прототипы интерфейсов,
продумываем структуру каталога,
прорабатываем интеграции

Разработка
Адаптивная верстка,
программирование

Запуск
Ваш сайт готов продавать

**Предпроектное
исследование**
Аудитория, конкуренты, интеграция

Дизайн
Рисуем все необходимые страницы
и интерфейсы в соответствии с
Вашим фирменным стилем

Тестирование
Проверка, наполнение контентом,
настройка аналитики, обучение

Создание веб-приложения, сайта

Frontend - разработка

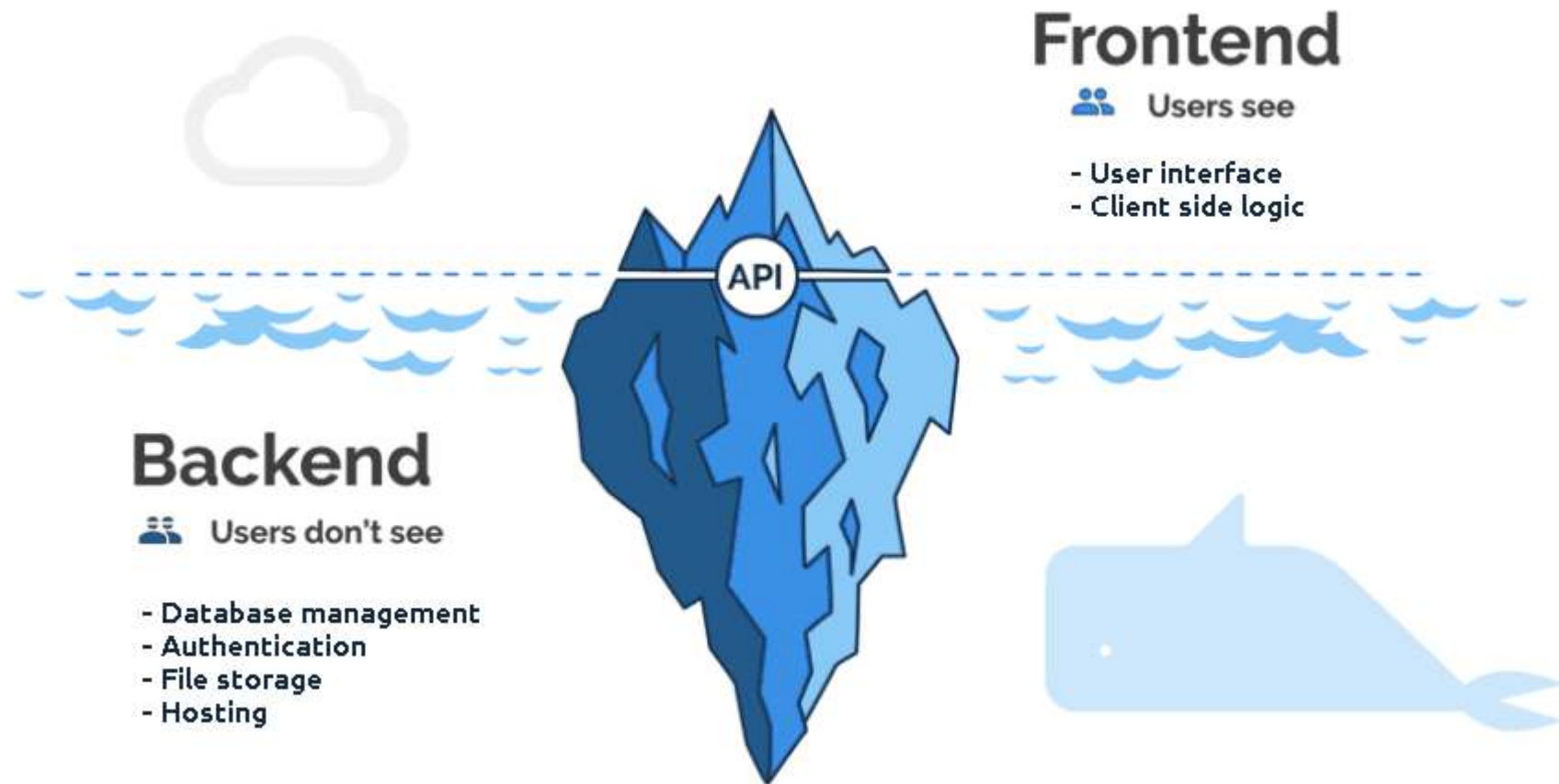
— это создание пользовательского интерфейса на клиентской стороне веб-сайта или приложения. Это всё, что видит пользователь, когда открывает веб-страницу, и с чем он взаимодействует: кнопки, баннеры и анимация

Backend - разработка

— это разработка бизнес-логики продукта (сайта или веб приложения). Отвечает за взаимодействие пользователя с внутренними данными, которые потом отображает frontend.

Это то, что скрыто от глаз пользователя и происходит вне его браузера и компьютера.

Создание веб-приложения, сайта



Создание веб-приложения, сайта

Фронтенд

Разработка пользовательского интерфейса. Создание и расположение всех элементов, с которыми взаимодействует пользователь: кнопок, форм, меню, слайдеров, галереи обоев — и далее по списку.

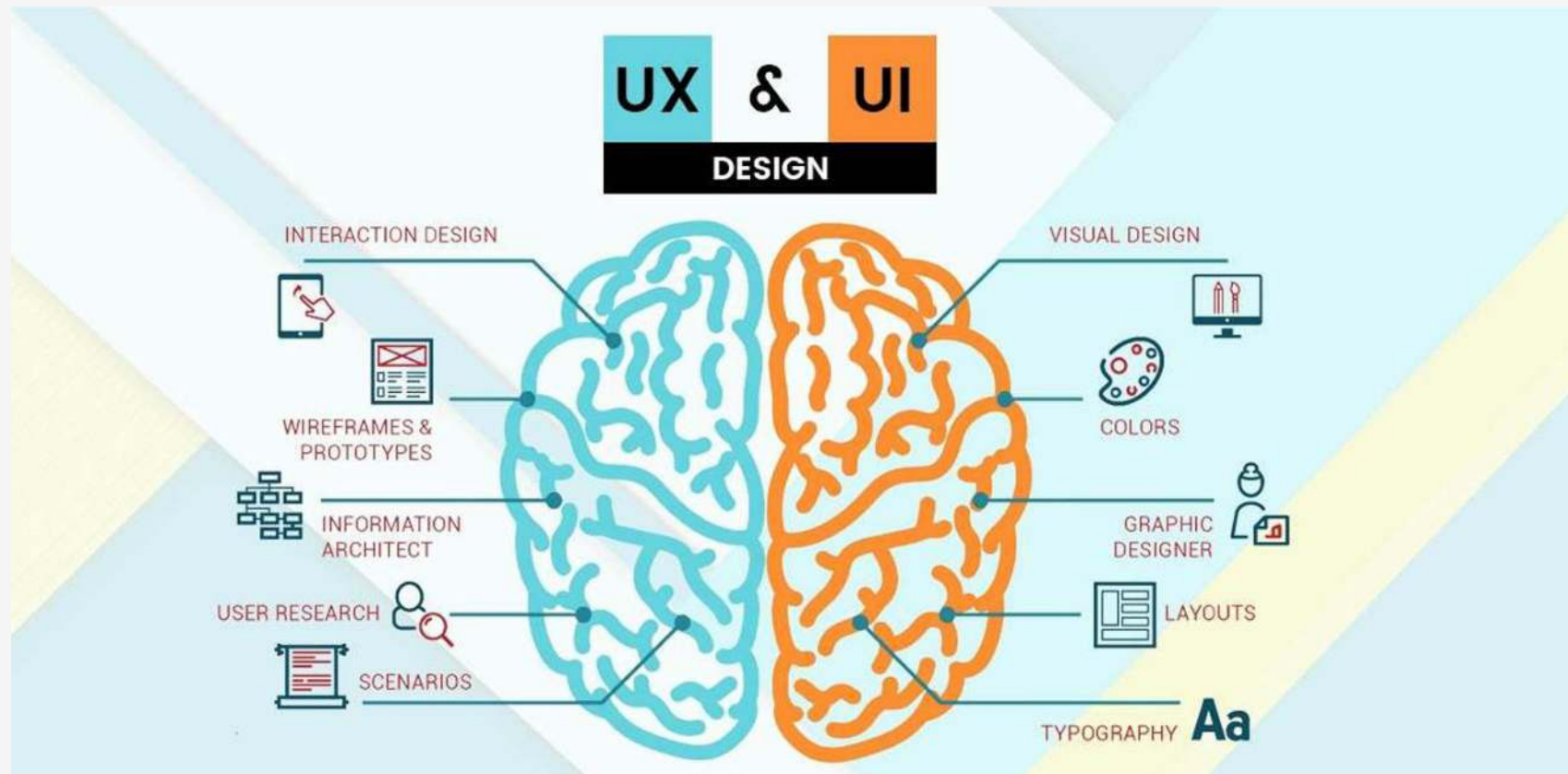
Вёрстка. Стилизация элементов на странице, чтобы информация отображалась правильно.

Интерактивность — оживляет страницы с помощью интерактивных элементов, анимаций и скриптов обработки действий пользователя.

Адаптивность. Сайт должен быть «гибким», чтобы он хорошо отображался на различных устройствах и экранах.

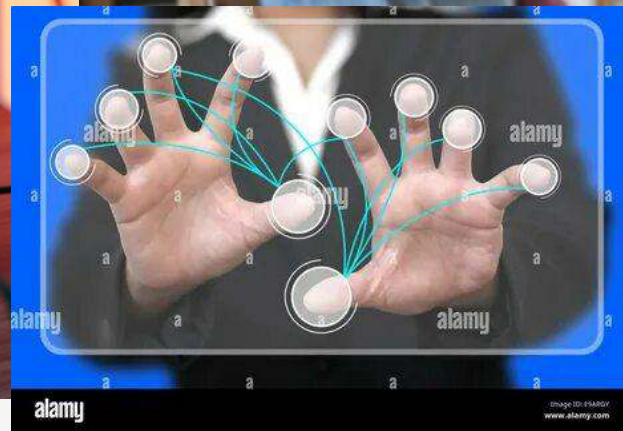
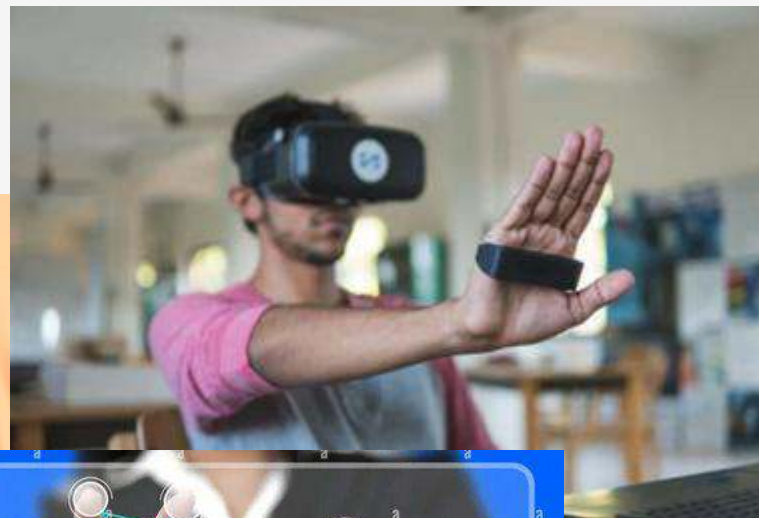
Тестирование и оптимизация. Тестирование работоспособности интерфейса, исправление ошибок и оптимизация производительности для быстрой загрузки страниц.

UI & UX (User Interface & User Experience)



Типы пользовательского интерфейса

- Текстовые
- Тактильные
- Голосовые
- Жестовые
- Нейронные
- **Графические**

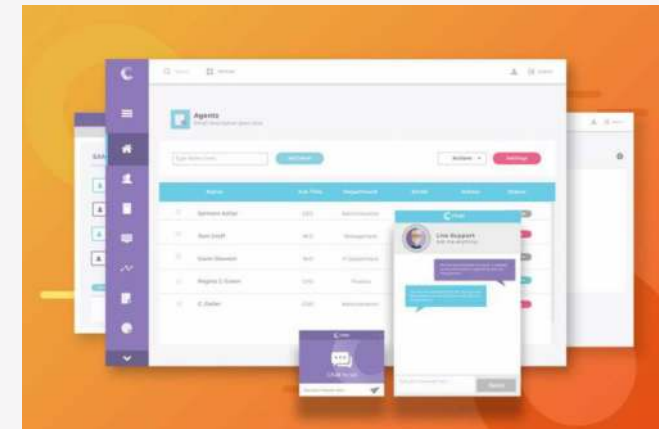


Графический интерфейс пользователя

Графический интерфейс

— тот, в оформлении которого задействуют все графические средства: цвета, стили и формы. Им обладает большинство сайтов и приложений и пользуются на смартфоне, планшете или компьютере. В зависимости от того, где и для чего используется графический интерфейс, различают:

- Мобильные интерфейсы
- Веб-интерфейсы
- Игровые интерфейсы



Мобильные интерфейсы

ОСОБЕННОСТИ

A diagram with a central title 'ОСОБЕННОСТИ' in purple. Below it, a purple curved line branches out to two sub-headers: 'Крупные элементы интерфейса' on the left and 'Расположение элементов' on the right. Both sub-headers and their corresponding text blocks are in purple.

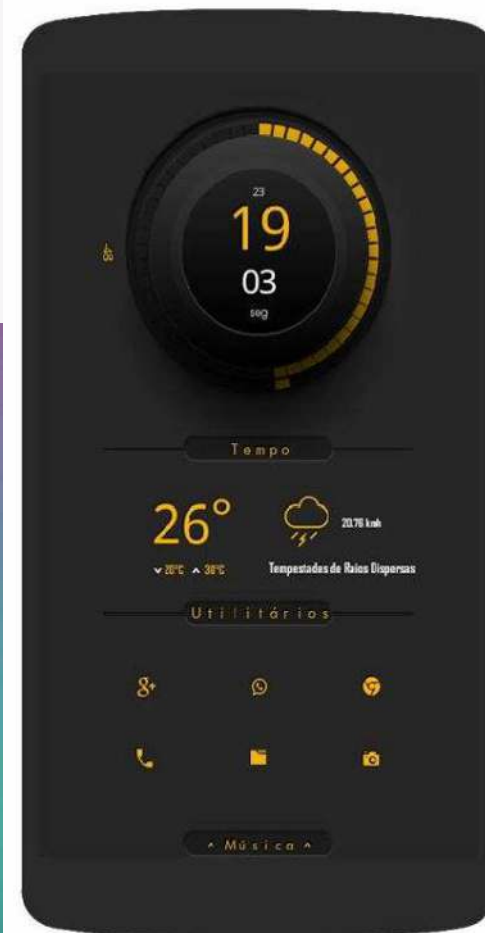
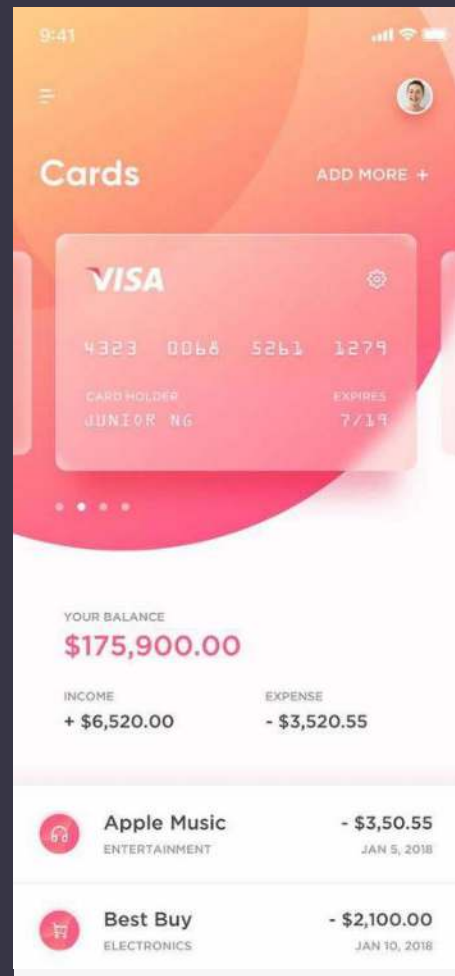
Крупные элементы интерфейса

блоки меню, кнопки или иконки — которые заполняют экран полностью: так, чтобы их было хорошо видно даже при маленьком разрешении.

Расположение элементов

обосновано анатомически: так, чтобы ими было удобно управлять с помощью одной руки, а пальцы не промахивались мимо кнопки.

Мобильные интерфейсы



Веб-интерфейсы

Это интерфейсы сайтов и онлайн-сервисов, которые работают в десктопных браузерах. От мобильных их отличает то, что масштаб и композиция элементов здесь могут быть любыми, а управляют ими с помощью мыши или тачпада. В таких графических интерфейсах чаще встречаются большие текстовые блоки, видео и анимация



Игровые интерфейсы

Созданы так, чтобы игрок сразу понимал, куда нужно двигаться и какие возможности у него есть. Главное место здесь отводится персонажам игры и вспомогательным средствам: машины, роботы или оружие. Текстовые блоки минимальные и используются только в качестве подсказок. Элементы меню часто стилизованы под сюжет игры



World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#) , [Policy](#) , November's [W3 news](#) , [Frequently Asked Questions](#) .

[What's out there?](#)

Pointers to the world's online information, [subjects](#) , [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#) ,X11 [Viola](#) , [NeXTStep](#) , [Servers](#) , [Tools](#) , [Mail robot](#) , [Library](#))

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#) , etc.

Самый первый сайт (23 августа 1991 года)

<https://info.cern.ch/hypertext/WWW/TheProject.html>

The World's Worst Website Ever!

COMING SOON: An even worse site! Look for it sometime later, eventually! we PROMISE!

Traffic Stats

SELF SERVE

THIS WEEK

999 9/10

LAST WEEK

999 9/10

DAILY RECORD

1.4058

>>>> You've found

The World's Worst Website **Ever!**

TWWWE is a project to highlight errors in web design by breaking every single design rule imaginable. [Click here](#) for an incomplete list of errors found on this site.

WATCH OUT! This site is under construction! =>

ForexClub

Как заработать на Forex?

Узнайте торговле на финансовом рынке с \$5000 на сайте!

Открыть учебный сайт

<-- send us email!

MAIN MENU

Click here to go to the main menu

[NEW TO THE SITE](#) [YAHOO NEWS](#) [WEATHER](#) [GOOGLE MAPS](#) [GREAT WEB DESIGN](#)
[ABOUT TWWWE](#) [VISIT KENTUCKY LAKE!](#) [SHOCKING PICTURES](#) [AMAZING COLOURS](#) [SMOKY MOUNTAINS!](#)

BREAKING NEWS: 3 out of 4 people say this is a terrible website!

WHAT'S NEW

Welcome to our new world's Worst Website EVER! we're stoked to be able to bring u da best in REALLY BAD WEBSITE DESIGN! Our goal is to break every single rule in website design!!! As an educational tool, the aim of this sight is to pinpoint and hi-light the very worst things I could do in designing a website. You can see a list of all the errors on this sight by [clicking here!](#)

We now have added a traffic marquee! Isn't it kewl??? LOL!

OHHH BOY!
v 2.0
of TWWWE coming SHORTLY!

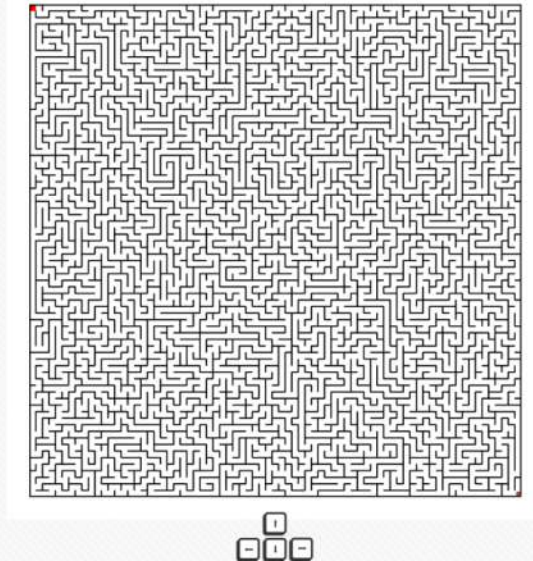
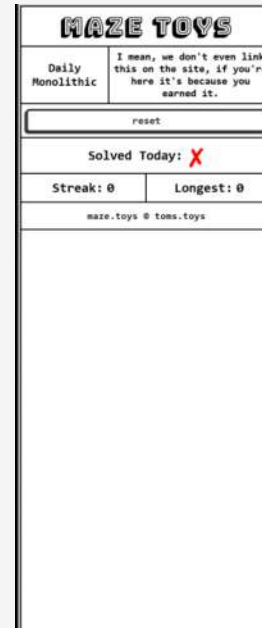
Как не надо...

The World's Worst Website Ever (сейчас недоступен). Был создан для демонстрации худших решений



LONG DOGE CHALLENGE

Your doge is **wow** long
You have collected 7 wows



0000001000



Как, возможно, можно

<https://theuselessweb.com/>

<https://cat-bounce.com/>

<https://longdogechallenge.com/>

3

GIT
GITHUB
GITHUB PAGES

Система контроля версия (СКВ)

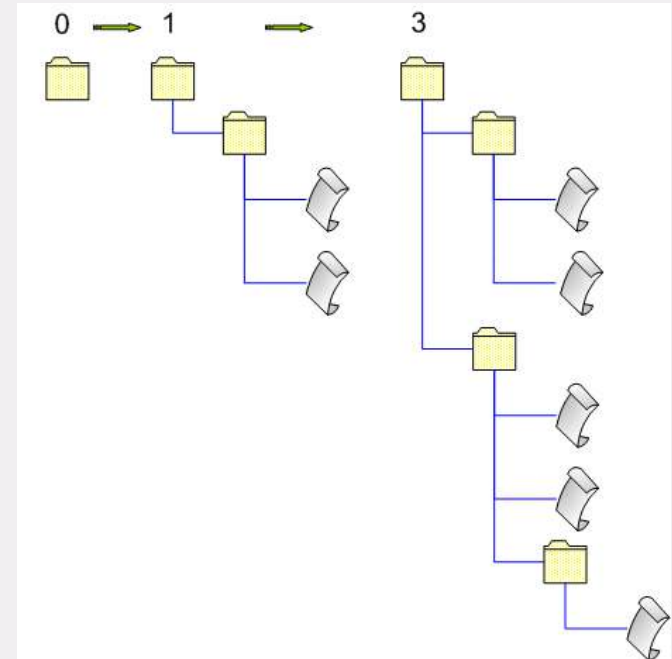
— это инструмент, который позволяет отслеживать изменения в файлах и координировать работу над проектом несколькими людьми. СКВ сохраняет историю изменений, что позволяет пользователям возвращаться к предыдущим версиям, сравнивать изменения и управлять конфликтами, возникающими при совместной работе.



Пример использования системы контроля версий

Предположим, команда разработчиков работает над проектом. Каждый разработчик может работать над своей частью кода, используя систему контроля версий, такую как Git.

Когда разработчик вносит изменения в код, он «коммитит» их в репозиторий. Система сохраняет текущее состояние файлов и добавляет описание изменений. Другие члены команды могут «получать» обновления из репозитория, видеть внесенные изменения и при необходимости объединять свои изменения с работой других. Если возникнет ошибка, команда сможет быстро вернуться к предыдущей стабильной версии кода.



Преимущества использования СКВ

1. Отслеживание изменений.
2. Возможность возврата к предыдущим версиям.
3. Совместная работа.
4. Управление конфликтами.
5. Легкость в экспериментировании.
6. Документация изменений.
7. Упрощение процесса развертывания.
8. Безопасность.
9. Аудит и контроль доступа.
10. Интеграция с другими инструментами.

Основные понятия СКВ

1

Рабочая директория

- папка, в которой хранятся все файлы проекта и в которой работает СКВ

2

Отслеживание файла

- файлы в рабочей директории могут быть добавлены под СКВ или нет. Это сделано для того, чтобы не отслеживать часто меняющиеся, но неважные для рабочего процесса файлы, например: объектные файлы, настройки сборщика и IDE, временные файлы, кэш и так далее

Основные понятия СКВ

3 Репозиторий

- хранилище, где сохраняются все версии файлов проекта.

Может быть локальным (на вашем компьютере) или удаленным (на сервере, например, на GitHub)

4 Коммит

- это снимок состояния файлов в репозитории в определенный момент времени. Каждый коммит содержит метаданные, такие как автор, дата и сообщение, описывающее изменения

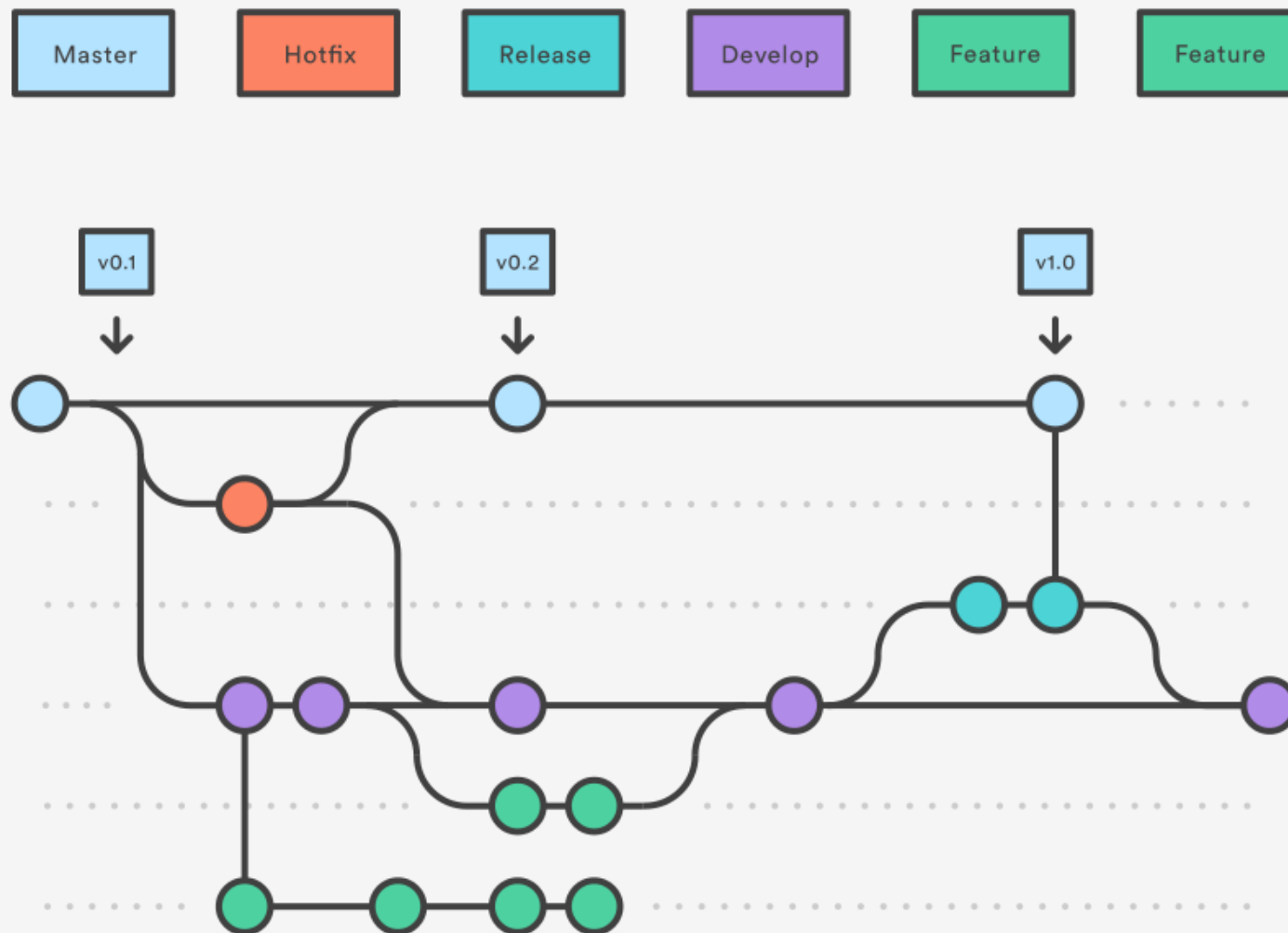


Основные понятия СКВ

5

Ветка

- это независимая линия разработки в репозитории. Основная ветка часто называется main или master. Ветки позволяют параллельно работать над различными функциями / исправлениями без вмешательства в основную кодовую базу



Основные понятия СКВ

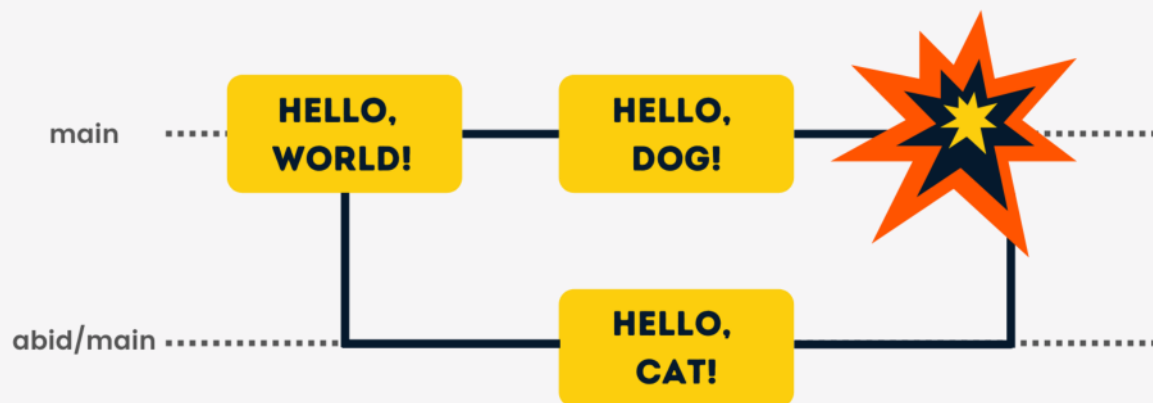
6 Слияние (Merge)

- процесс объединения изменений из одной ветки в другую.

Слияние позволяет интегрировать новые функции или исправления в основную ветку проекта

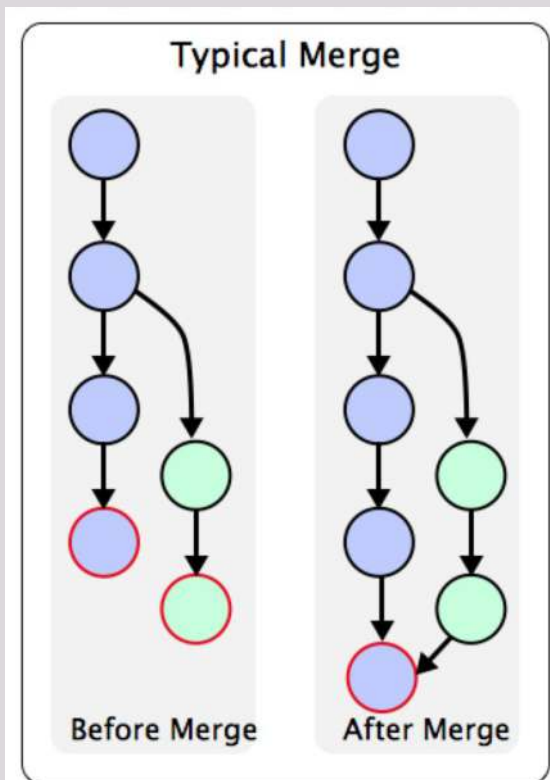
7 Конфликт

возникает, когда изменения в двух разных ветках затрагивают одни и те же строки кода. Пользователь должен вручную разрешить конфликт, выбрав, какие изменения сохранить



Основные понятия СКВ

6 Слияние (Merge)



8 Форк

- это копия репозитория, созданная пользователем для внесения изменений без воздействия на оригинальный проект. Форки часто используются для разработки новых функций или исправлений в открытых проектах

Основные понятия СКВ

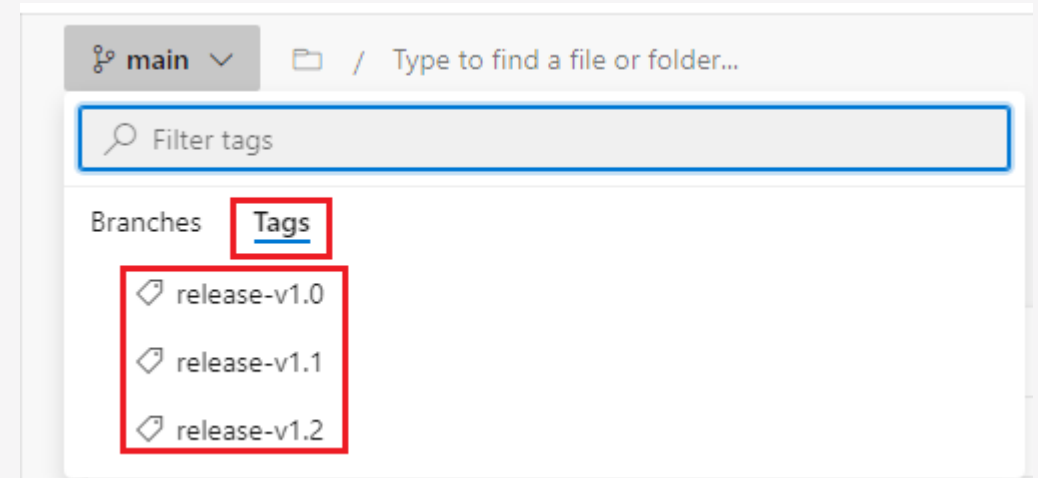
9 Пулл-реквест (Pull Request)

- запрос на слияние изменений из одной ветки/форка в другую.

Пулл-реквест позволяет другим участникам команды просмотреть и обсудить изменения перед их интеграцией.

10 Тег

это метка, присвоенная конкретному коммиту для обозначения важного момента в истории проекта (например, версия релиза). Теги обычно используются для создания стабильных версий программного обеспечения



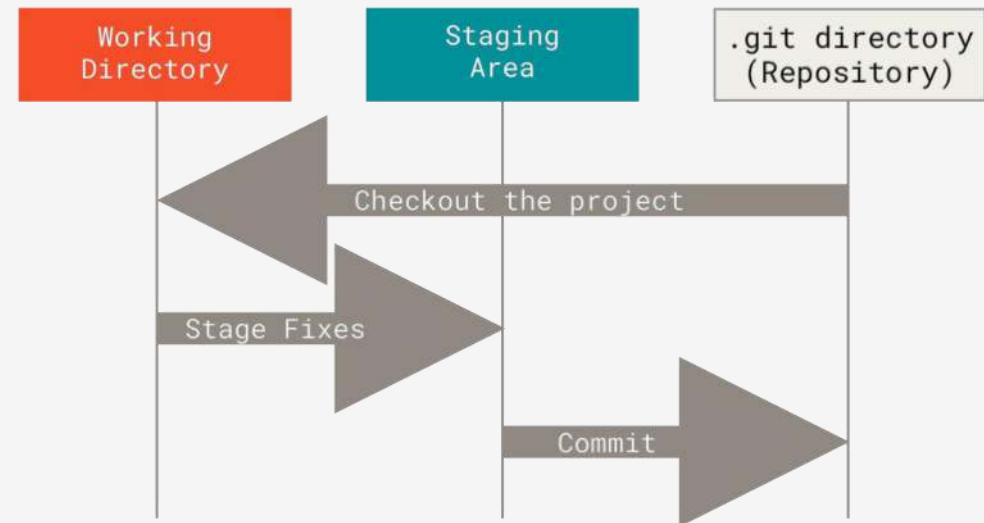
Основные понятия СКВ

11 История (Log)

- это последовательность всех коммитов в репозитории, которая позволяет отслеживать изменения и видеть, кто и когда вносил правки.

12 Рабочая копия (Working Directory)

- это локальная версия файлов проекта на вашем компьютере, где вы можете вносить изменения перед их коммитом в репозиторий.



Основные понятия СКВ

13 Состояние (Status)

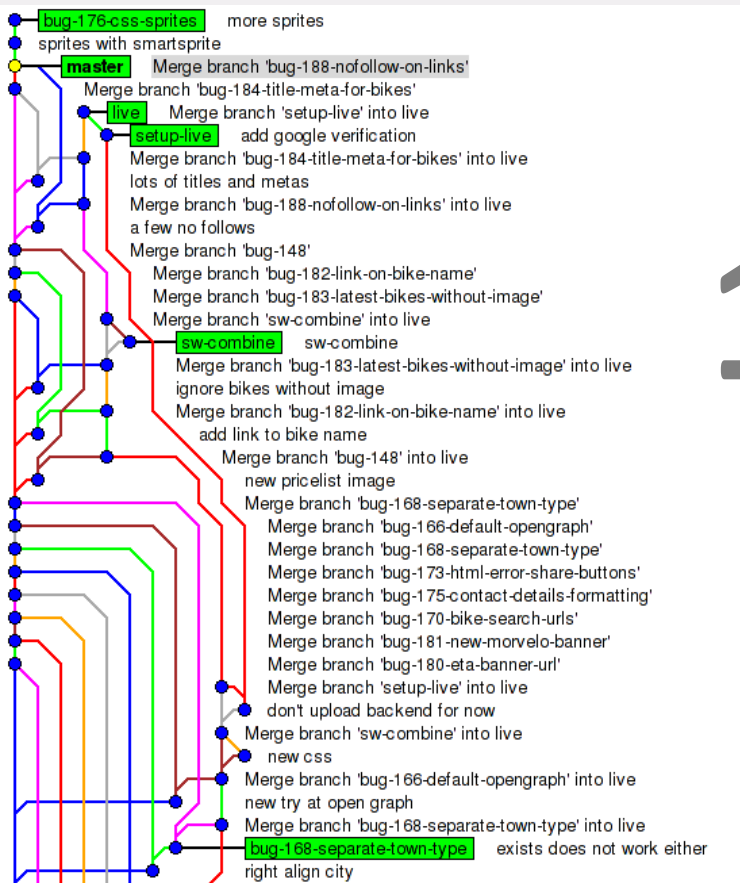
Команда `git status`, например, показывает текущее состояние рабочей директории и индекса, включая информацию о том, какие файлы были изменены, добавлены или удалены.

14 Отслеживаемые и неотслеживаемые файлы

Отслеживаемые файлы — это файлы, которые находятся под контролем СКВ и изменения которых отслеживаются.

Неотслеживаемые файлы — это те, которые не добавлены в систему управления версиями.

Как Git хранит состояния



Git использует уникальный подход к хранению состояний файлов и версии проекта. Рассмотрим основные аспекты того, как Git управляет состояниями

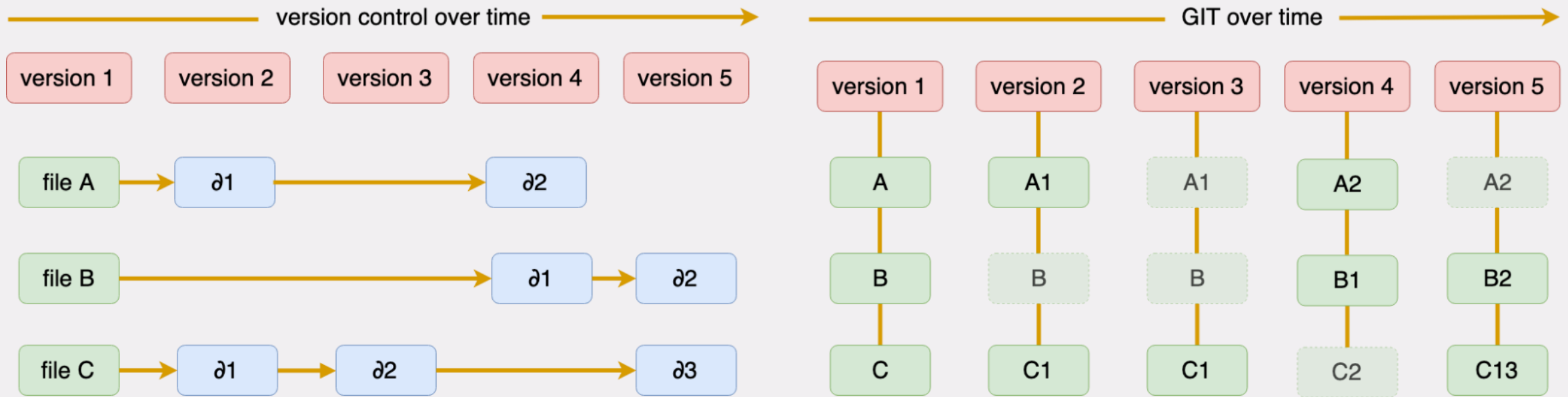
1 Снимки (Snapshots)

В отличие от других СКВ, которые могут хранить изменения в виде диффов (различий) между версиями, Git сохраняет состояние проекта как снимки (snapshots). Каждый раз, когда вы делаете коммит, Git сохраняет полную версию всех файлов в репозитории. Если файлы не изменились с момента последнего коммита, Git не дублирует их, а просто ссылается на уже существующую версию.

У состояния есть родитель

Как Git хранит состояния

В отличие от других СКВ, которые могут хранить изменения в виде диффов (различий) между версиями, Git сохраняет состояние проекта как снимки (snapshots).



Как Git хранит состояния

Патч

— это набор изменений, которые были внесены в файлы между двумя состояниями (например, коммитами). Патчи обычно представляют собой текстовые файлы, которые описывают, какие строки были добавлены, изменены или удалены в файлах.

Основные аспекты патча

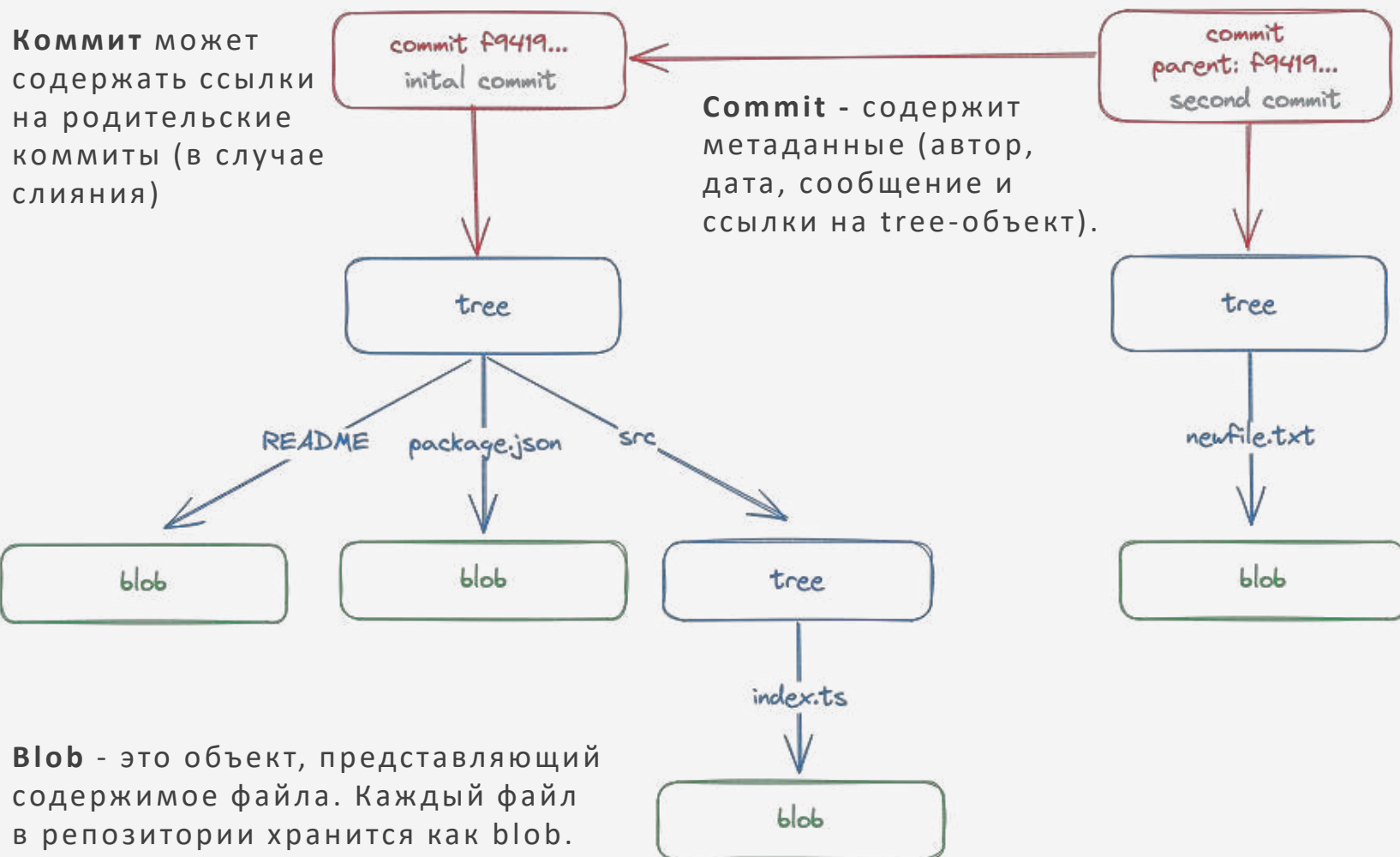
1. Обычно имеют текстовый формат, с информацией о файлах и изменениях. Так, они могут содержать строки, начинающиеся с + для добавленных строк и - для удаленных.
2. Могут использоваться для обмена изменениями между разработчиками.
3. Патчи можно применять к другим версиям проекта с помощью команды `git apply`, что позволяет интегрировать изменения без необходимости делать полный коммит.
4. Помогают разработчикам видеть, что именно изменилось в коде, что может быть полезно при код-ревью или при решении конфликтов слияния.

Как Git хранит состояния

2 Объекты

Git использует три основных типа объектов для хранения данных:

- Blob (Binary Large Object)
- Tree
- Commit. Это объект, представляющий состояние проекта на момент коммита.



Tree - это объект, который представляет собой директорию и содержит ссылки на blobs и другие tree-объекты. Он описывает структуру файловой системы.

Как Git хранит состояния

3 Хеширование

Git использует алгоритм SHA-1 для создания уникальных идентификаторов (хешей) для каждого объекта. Хеш создается на основе содержимого объекта, что обеспечивает целостность данных. Если содержимое файла изменится, изменится и его хеш.

4 Индекс (Staging Area)

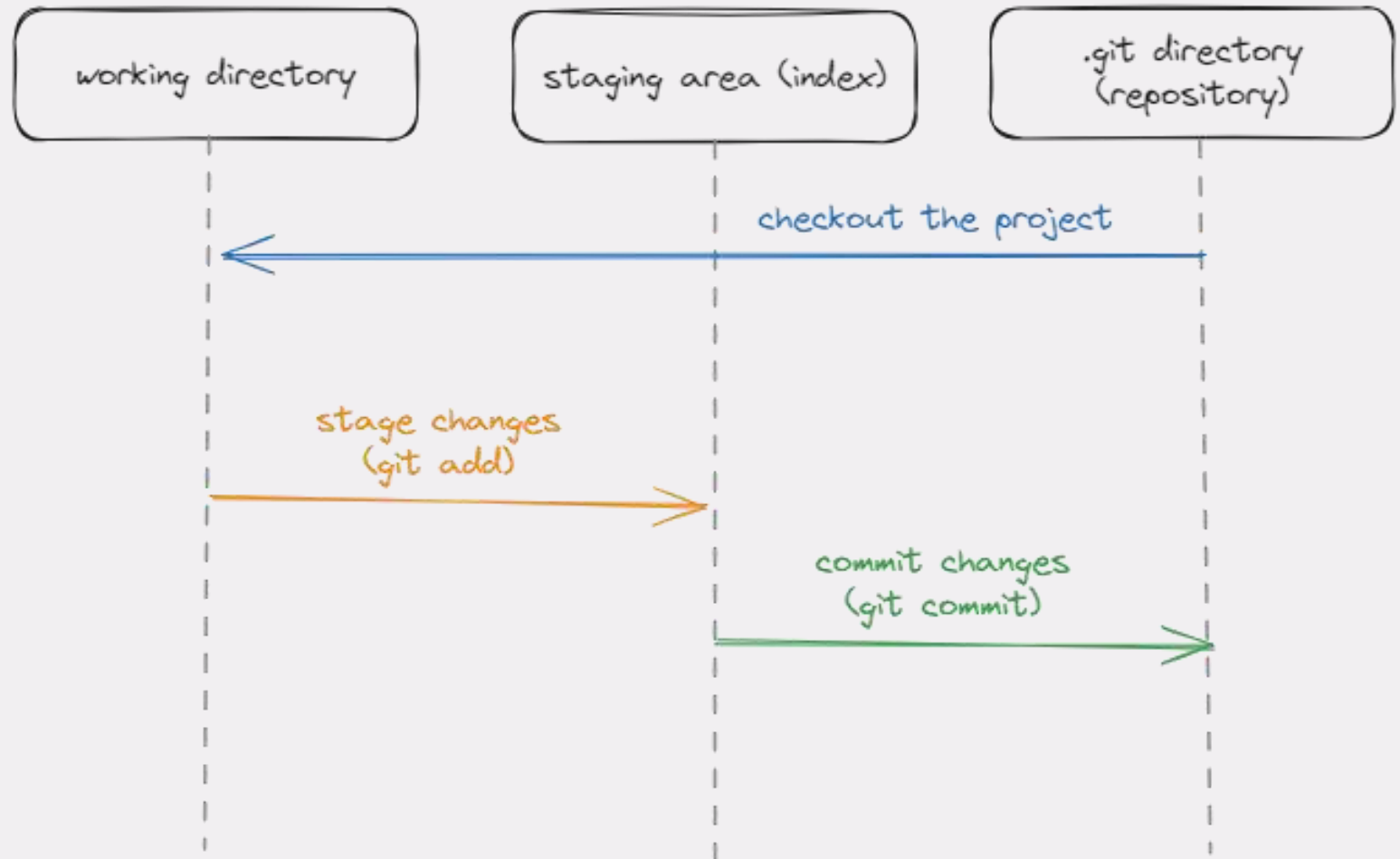
Перед тем как сделать коммит, изменения добавляются в индекс (или staging area). Индекс = промежуточное состояние, где вы можете подготовить файлы к коммиту. Вы можете выбрать, какие изменения хотите включить в следующий коммит.

Как Git хранит состояния

5 Рабочая директория

Это ваша локальная копия файлов проекта. Когда вы вносите изменения в файлы, они находятся в рабочей директории.

Чтобы сохранить эти изменения в репозитории, вам нужно добавить их в индекс и затем сделать коммит.



Как Git хранит состояния

6 Ветки

представляют собой указатели на конкретные коммиты. Каждая ветка хранит ссылку на последний коммит в этой ветке, что позволяет легко переключаться между различными версиями проекта и работать над несколькими функциями одновременно.

7 История

Git хранит историю коммитов в виде графа, где каждый коммит может ссылаться на один или несколько родительских коммитов. Это позволяет отслеживать изменения и видеть, как проект развивался со временем.

8 Удаленные репозитории

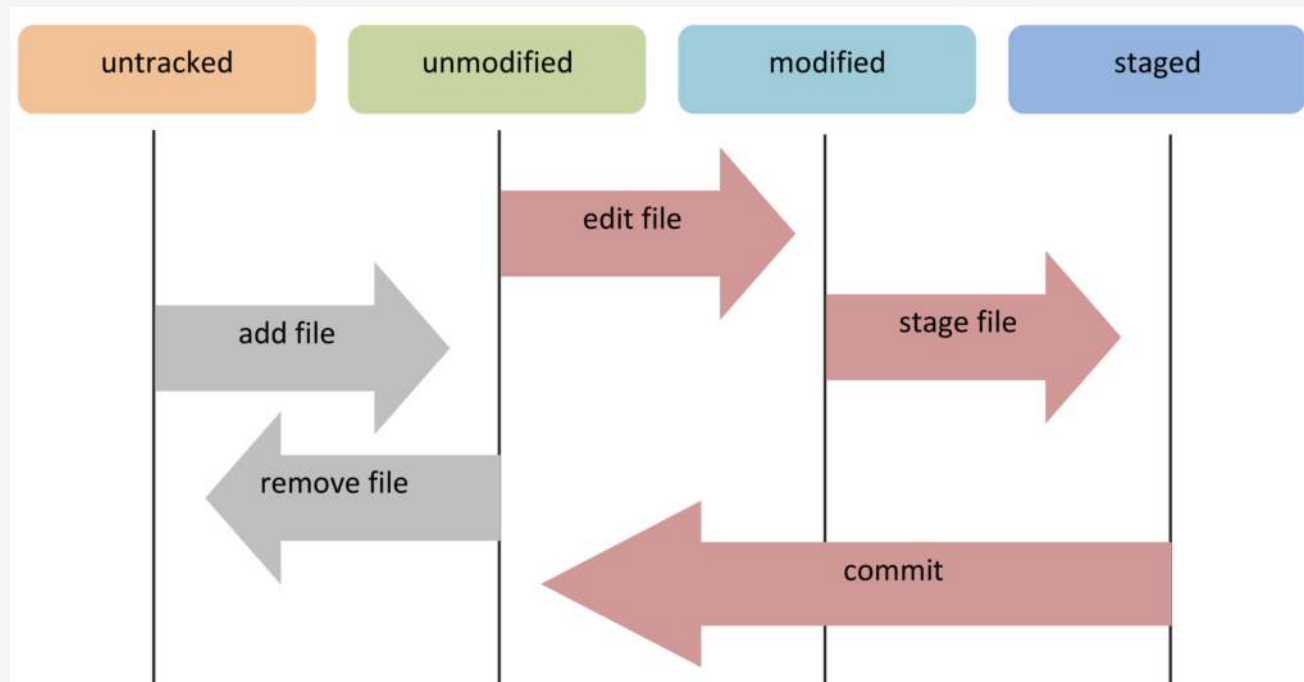
Git поддерживает работу с удаленными репозиториями, что позволяет синхронизировать локальные изменения с центральным хранилищем (например, на GitHub). Изменения могут быть загружены (push) или загружены (pull) из удаленного репозитория.

Жизненный цикл файла в Git

В Git различают три области:

- историю зафиксированных состояний;
- текущее состояние рабочей директории;
- индекс.

Рабочая директория - то, что мы видим в окне проводника. Мы работаем с файлами в ней, можем их редактировать любыми программами как обычно.



Индекс - набор файлов рабочей директории, которые отслеживаются СКВ. Когда мы делаем коммит, содержимое индекса записывается в историю состояний.

Если мы создадим файл в рабочей директории, он не будет отслеживаться СКВ. Это сделано, чтобы не вычищать постоянно мусор из служебных файлов.

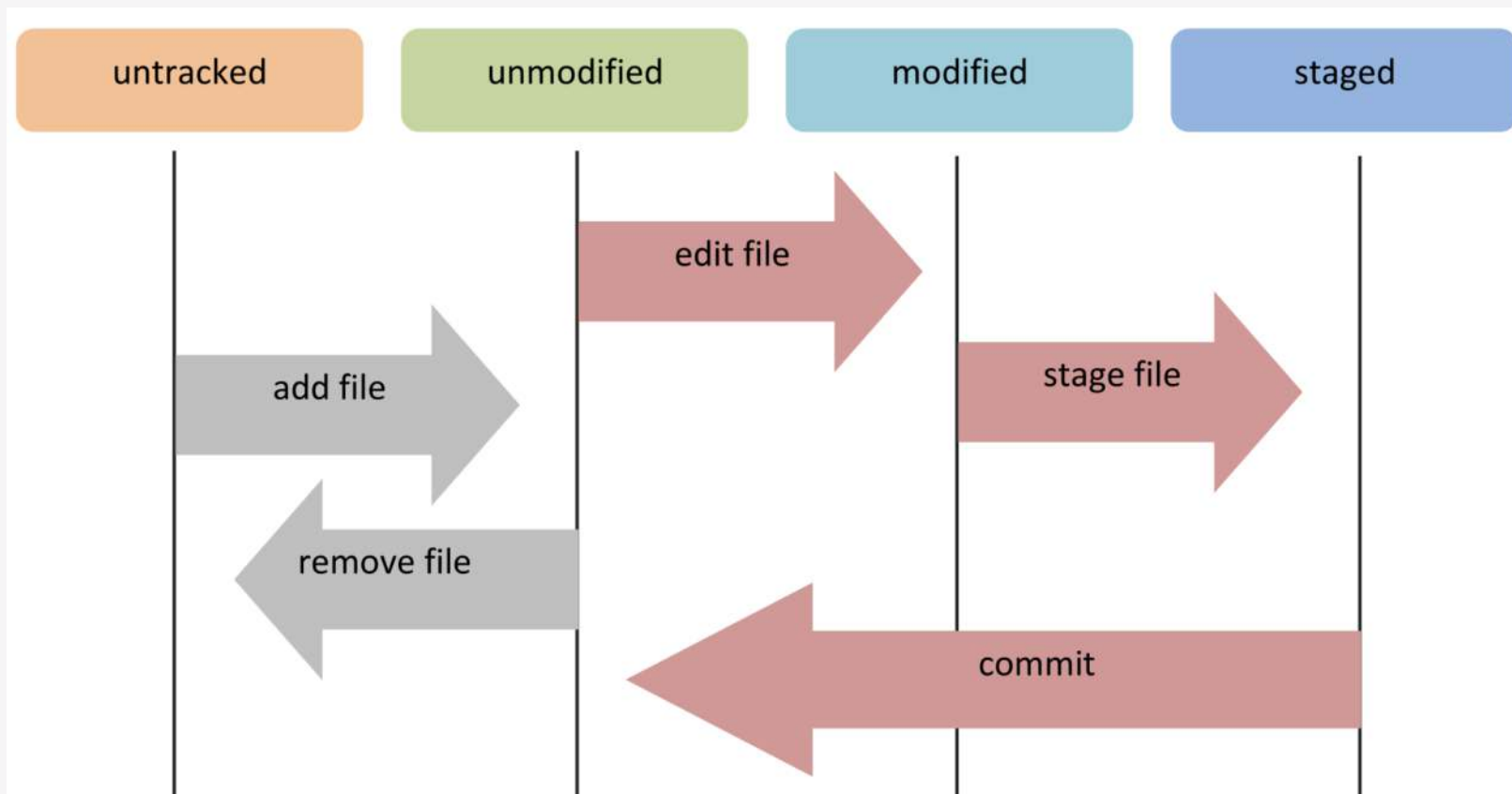
Жизненный цикл файла в Git

Файл можно добавить под СКВ командой **git add**.

Далее будем говорить только о файлах, отслеживаемых СКВ.

Если мы изменили файл в рабочей директории, он становится **модифицированным** по сравнению с последним коммитом.

Когда мы делаем коммит, мы записываем **новое состояние** и все файлы опять становятся **неизмененными**.



Во многих графических средах состояние файла отображается цветом для удобства.

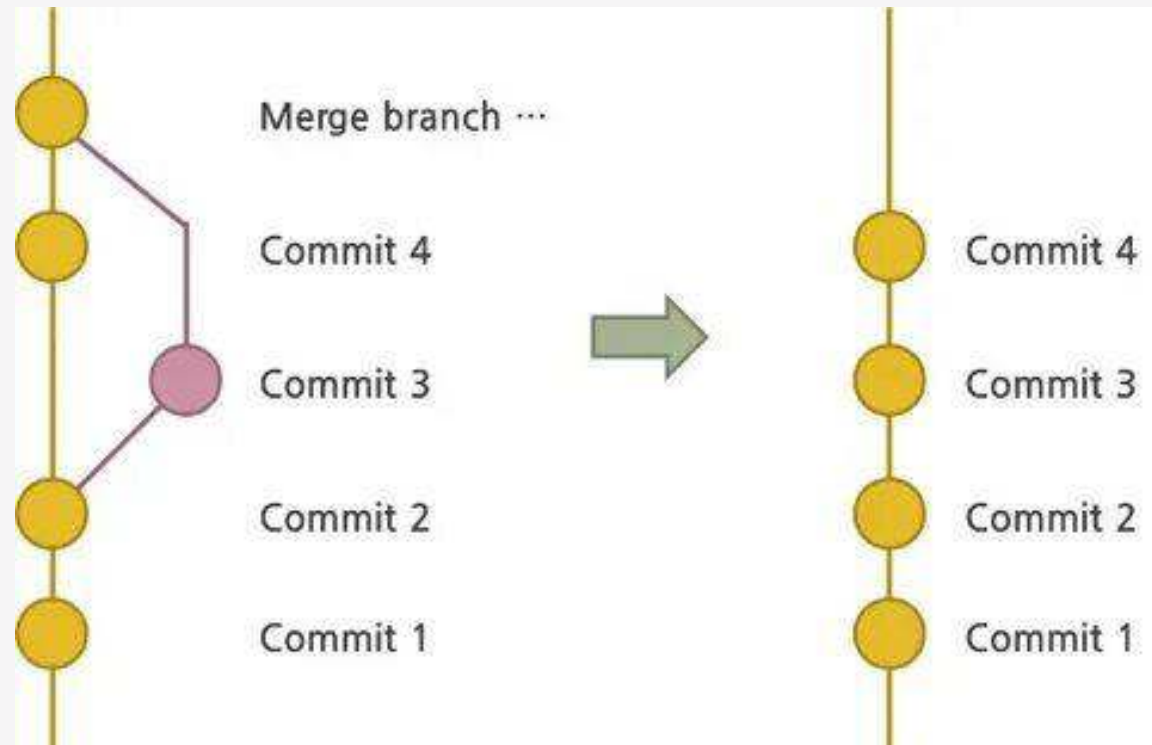
Ветки

Ветки - это специальные указатели на коммиты, которые перемещаются с каждым новым коммитом.

Ветвление - способ изолировать одни изменения от других.

Например, разработка одной фичи независимо от хода работ над другой.

Что такое ветки и зачем они нужны?



Git позволяет работать с ветками комфортно и, при необходимости, объединять их.

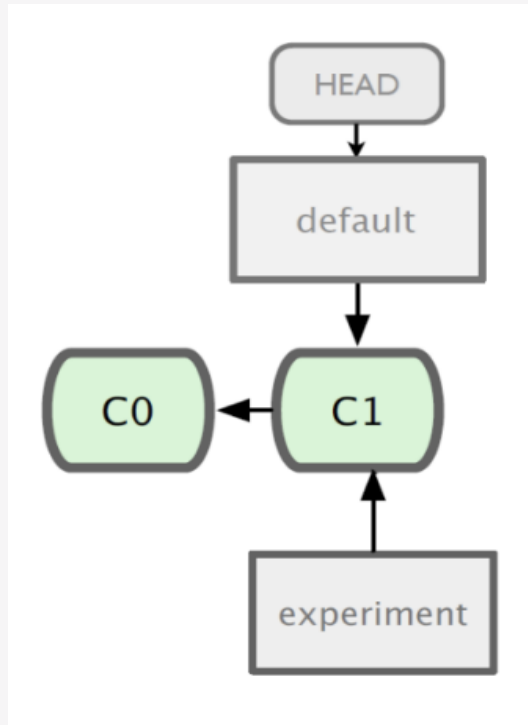
Ветки

Создать новую ветку
можно командой
git branch <name>

Команда **git branch**
выводит все ветки.

```
$ git branch
* default
  experiment
```

Создание новой ветки



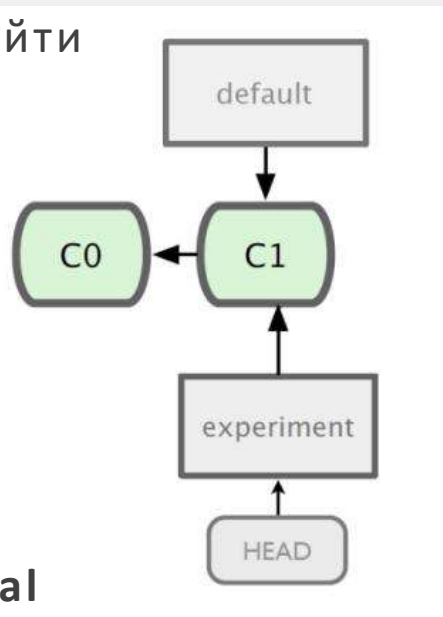
git branch experimental

Ветка по умолчанию по соглашению
называется master
(можете также столкнуться
с названием main)

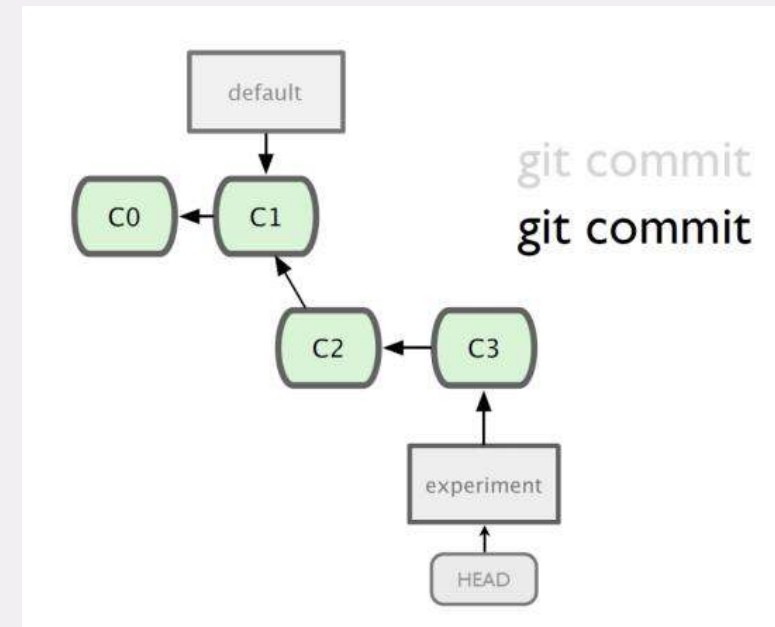
Работа с ветками

Git хранит специальный указатель HEAD - он показывает, на какой ветке вы сейчас находитесь. Чтобы перейти на существующую ветку, вам надо выполнить команду **git checkout**

git checkout experimental



Ветка, на которую указывает HEAD, движется вперёд с каждым коммитом. Теперь коммиты, которые мы будем делать, будут относиться к новой ветке

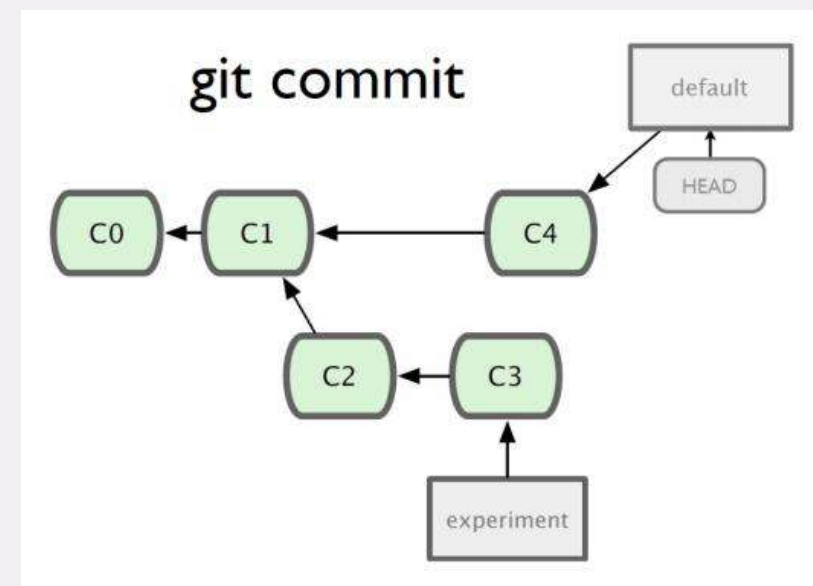
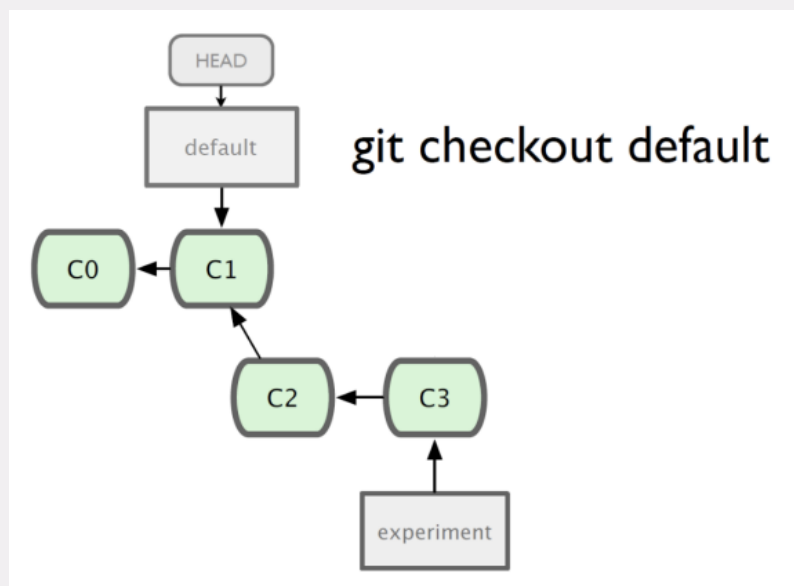


Работа с ветками / Переключение между ветками

При переходе в другую ветку состояние рабочей директории приводится в соответствие с состоянием последнего коммита в этой ветке. Поэтому все изменения будут основываться на том состоянии.

Переход между ветками откатывает изменения, которые вы делали в другой ветке, но не удаляет их. Все фиксированные изменения сохраняются в истории веток.

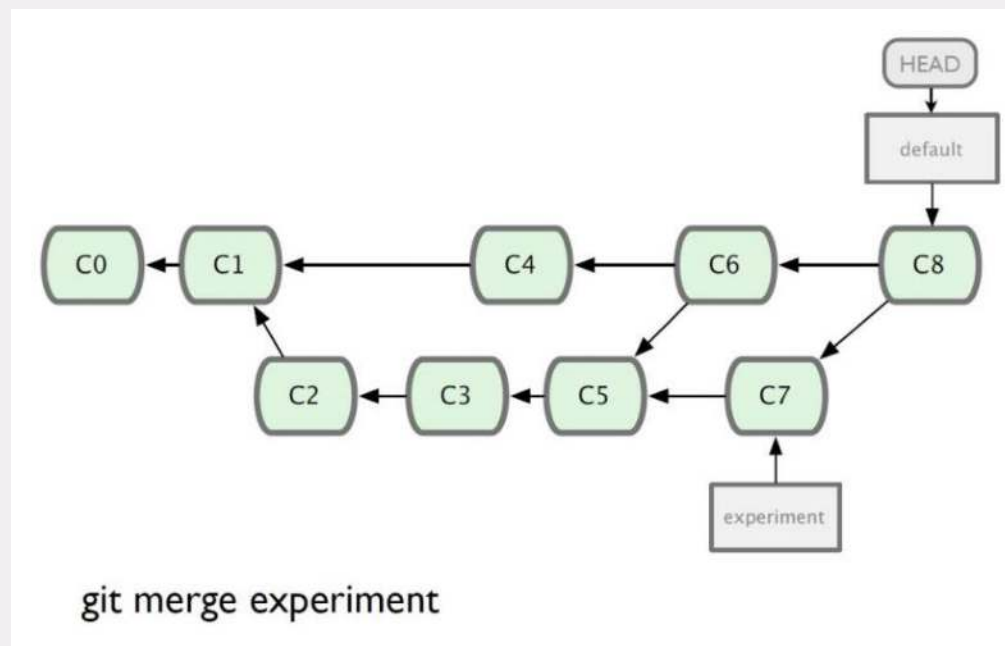
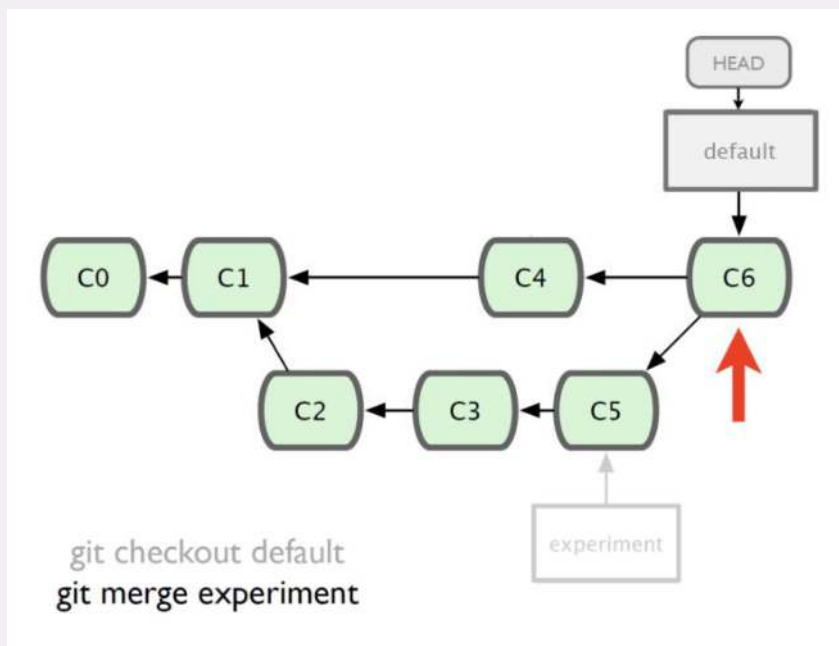
При использовании веток история коммитов уже не будет линейной



В любой момент можно перейти в другую ветку и начать коммитить в нее.

Работа с ветками / Объединение веток

Две ветки можно объединить (слить). Это значит, что мы применяем к файлам изменения, которые произошли и в одной ветке и во второй. Команда **git merge <branch>** сливает указанную ветку в текущую. Это значит, что создастся новый коммит в текущей ветке, учитывающий изменения в указанной.



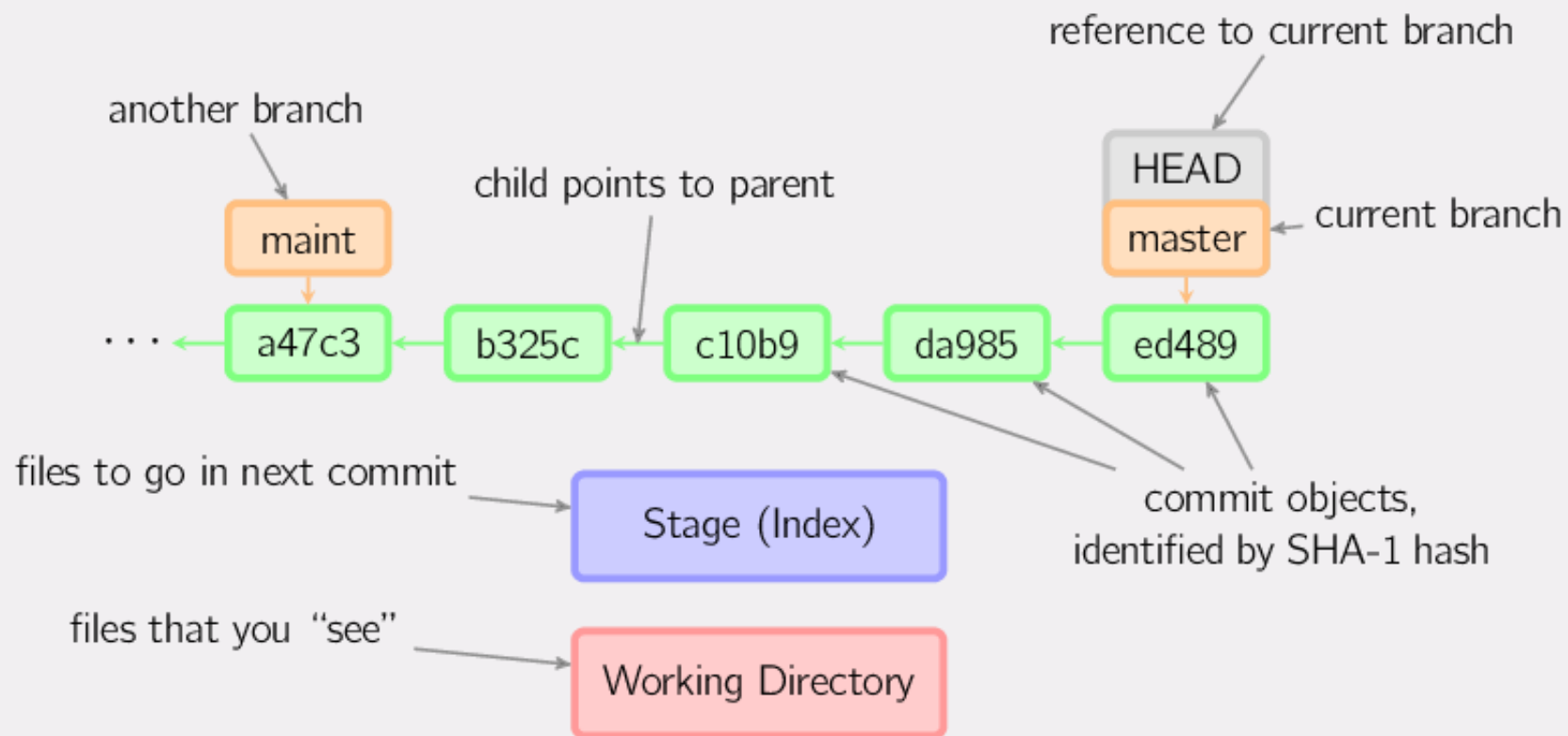
Коммиты слияния имеют двух родителей.

Git автоматически определяет наилучшего общего предка для слияния. После слияния ветка не уничтожается, над ней можно продолжать работать

Команды Git

Иллюстрации имеют единую цветовую схему:

- Коммиты — зелёные, с 5-ти буквенными идентификаторами и стрелками к родителям.
- Ветки — оранжевые, указывающие на коммиты.
- Специальная ссылка HEAD показывает текущую ветку.



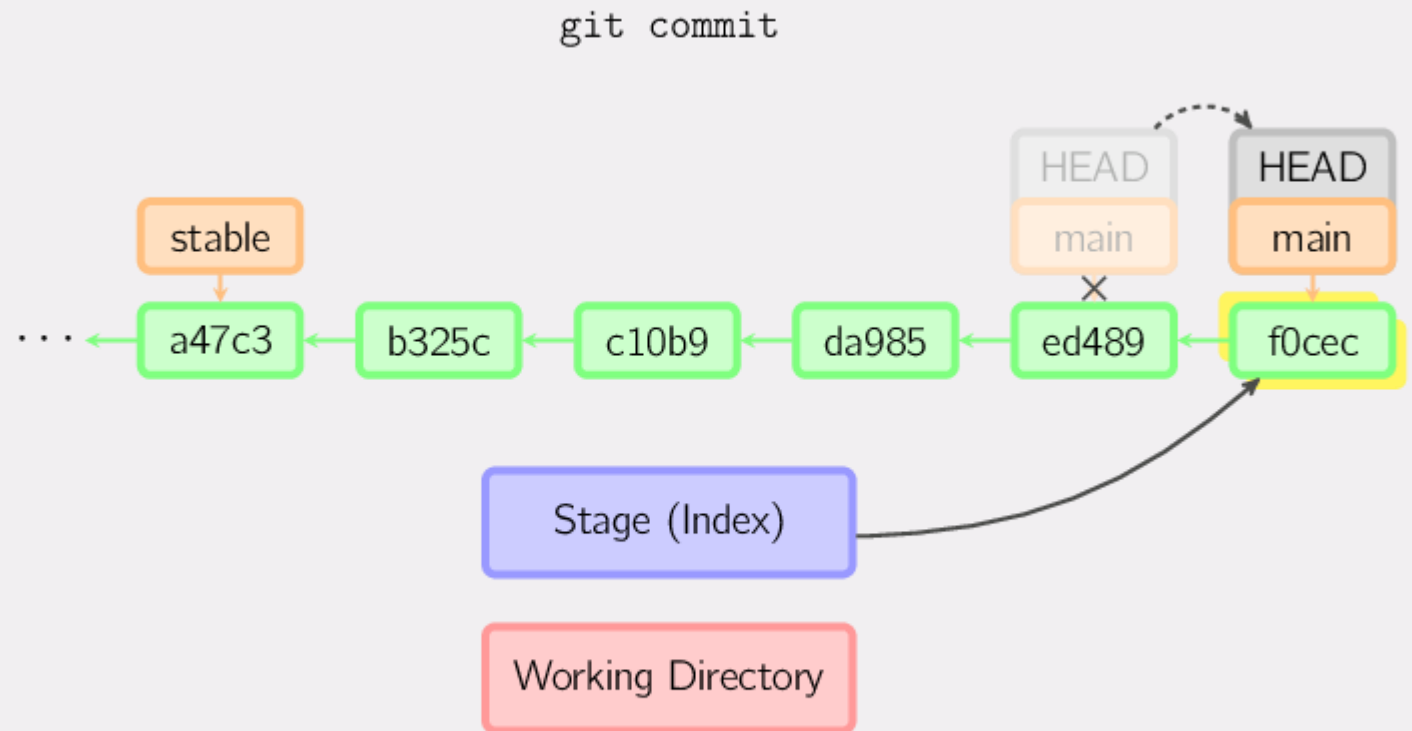
Текущая ветка `main` указывает на этот коммит, а ветка `stable` указывает на его предка

На иллюстрации отображены последние пять коммитов, последний из которых имеет хеш `ed489`

Команды Git

При выполнении коммита git **создаёт новый объект коммита** на основе файлов со сцены, а текущий коммит становится его родителем.

Указатель текущей ветки перемещается на новый коммит. Например, после добавления нового коммита f0cec, родителем которого стал ed489, ветка main указывает на f0cec.

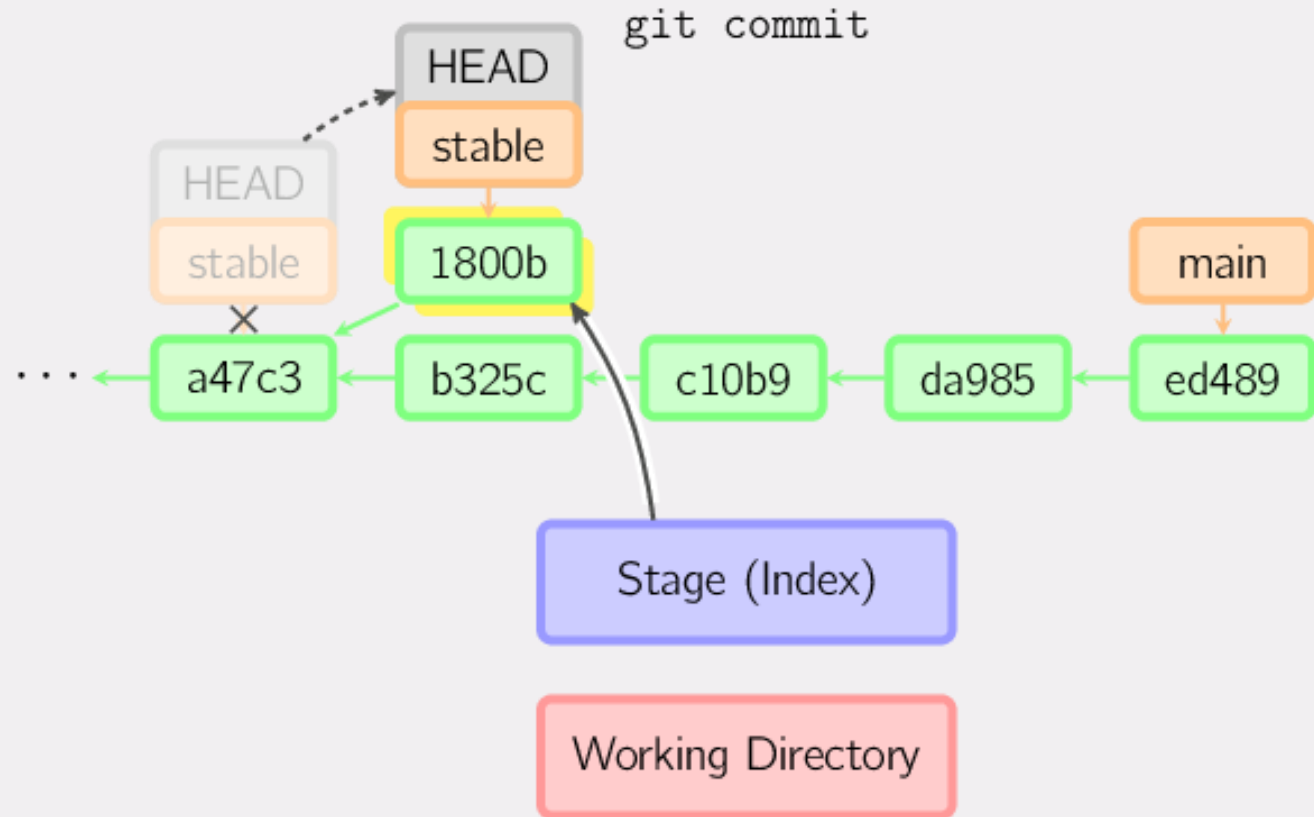


Команды Git

Если одна ветка является предком другой, например, ветка `stable` с новым коммитом `1800b`, то она больше не будет предком ветки `main`.

Для объединения изменений из разделённых веток

используйте команды `merge` или `rebase`.



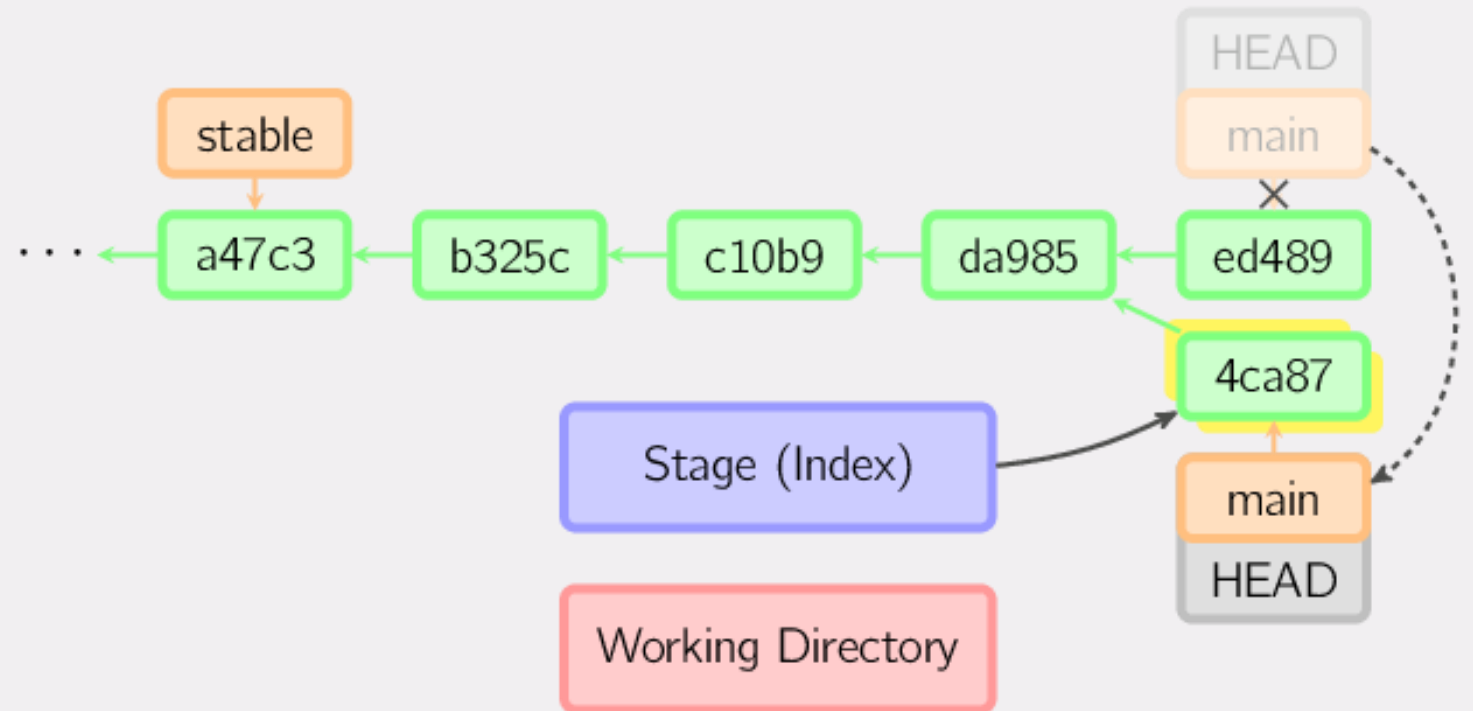
Команды Git

Ошибку в последнем коммите
можно исправить с помощью

git commit --amend,

что создаёт новый коммит
с родителем ошибочного коммита

`git commit --amend`



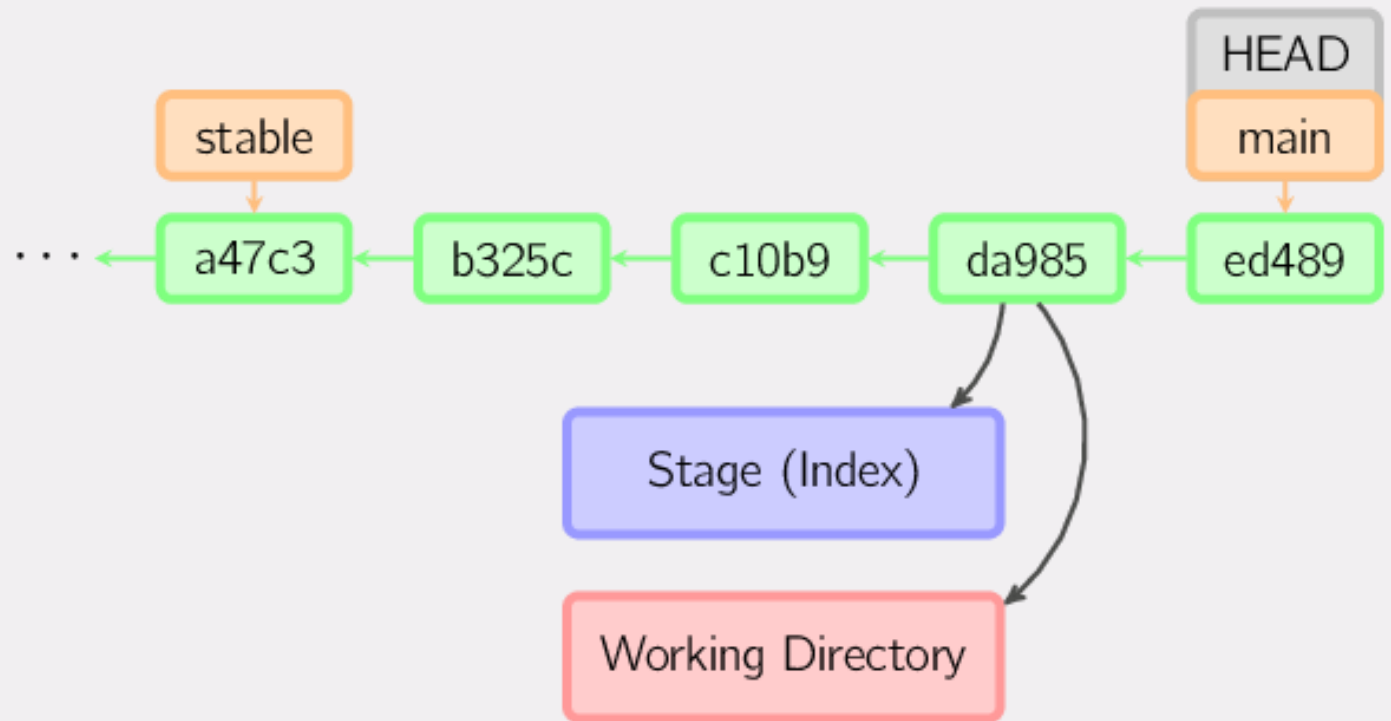
`git commit --amend`

Команды Git

Команда `checkout` копирует файлы из истории или сцены в рабочую директорию и позволяет переключаться между ветками.

При указании имени файла `git` копирует его из указанного коммита на сцену и в рабочую директорию (например, `git checkout HEAD~ foo.c`)

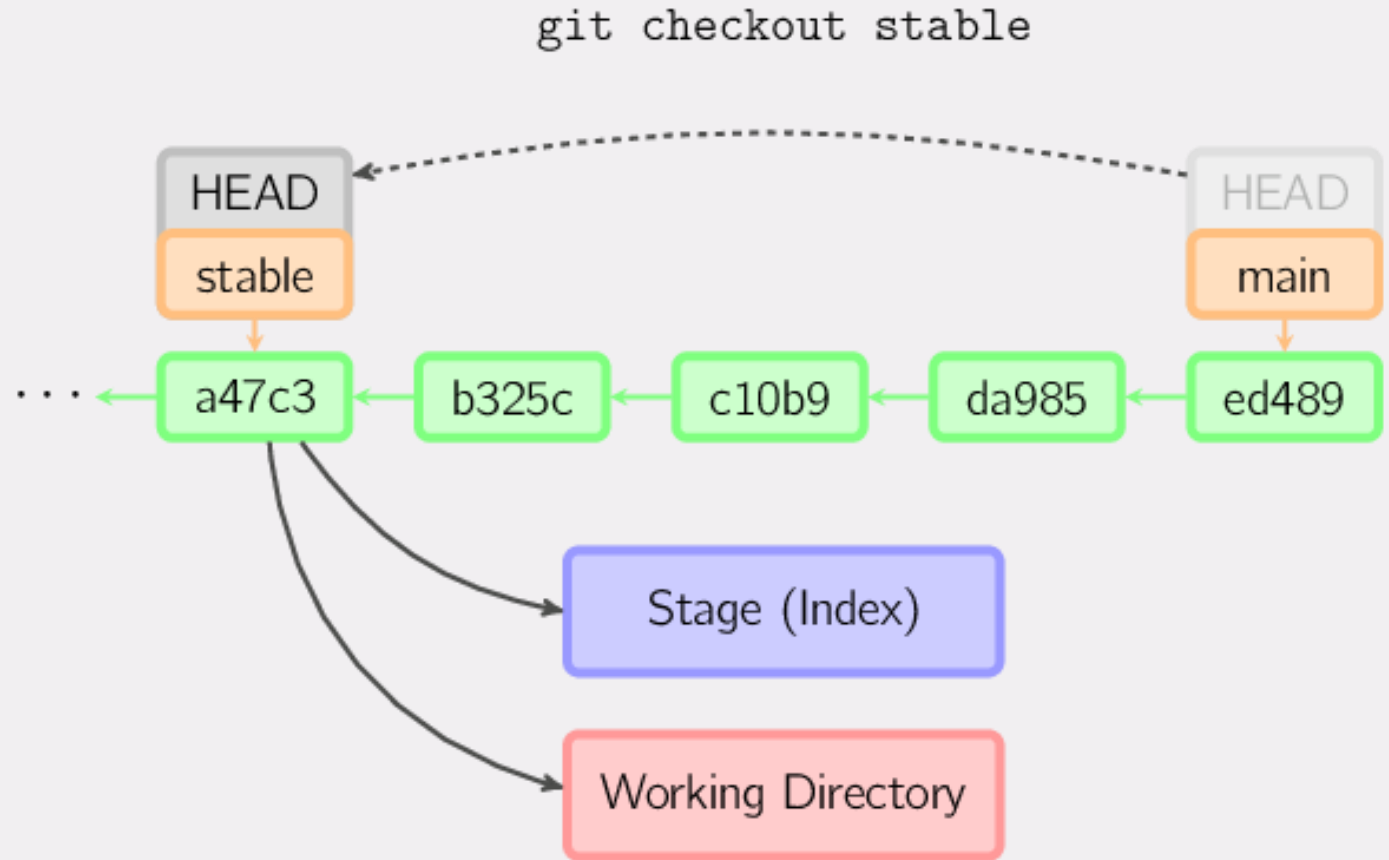
`git checkout HEAD~ files`



Команды Git

Если указано имя локальной ветки, указатель HEAD перемещается на неё, а сцена и рабочая директория обновляются в соответствии с новым коммитом.

Любой файл, который присутствует в новом коммите (a47c3 ниже), будет скопирован из истории; любой файл, который был в старом коммите (ed489), но отсутствует в новом, будет удалён; любой файл, который не записан ни в одном коммите, будет проигнорирован

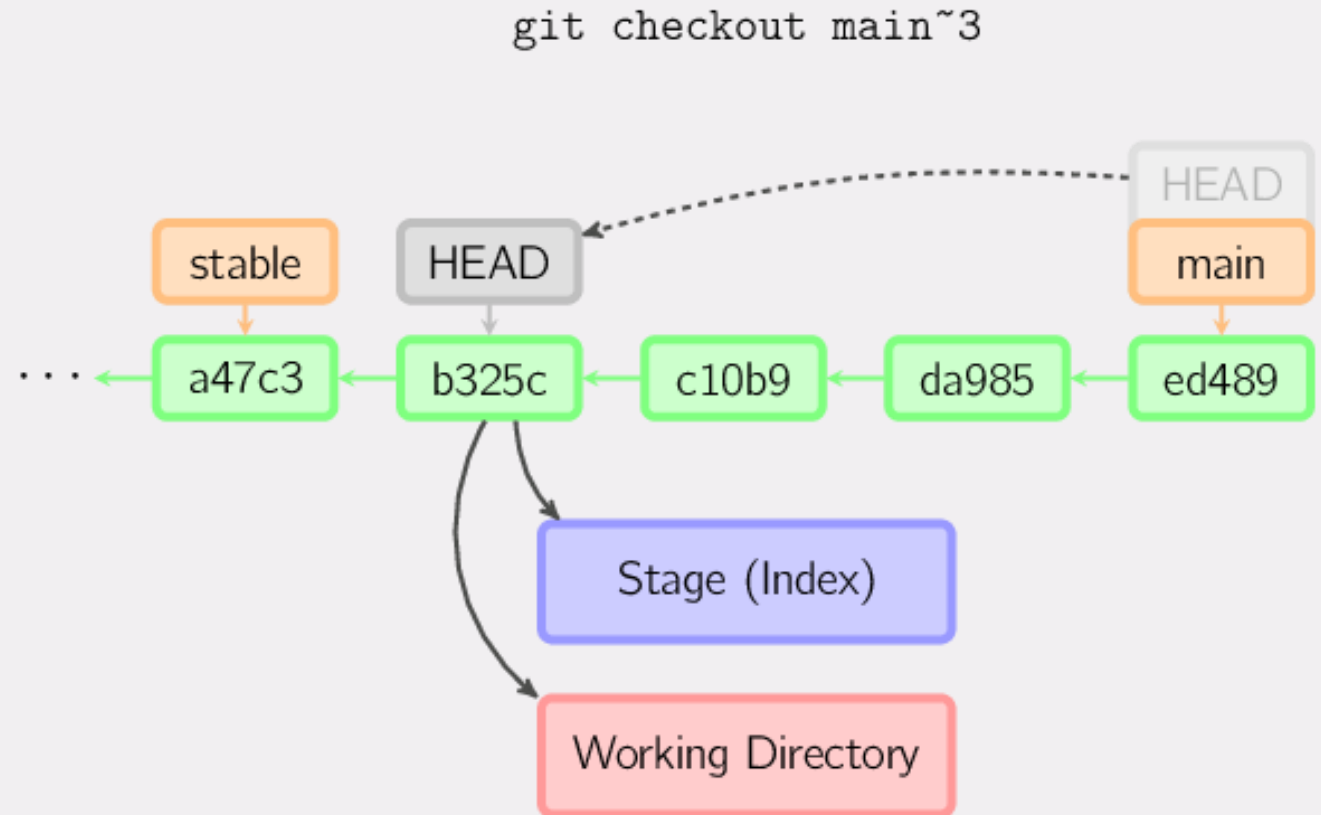


Команды Git

При отсутствии имени файла и ветки, но наличии тега или SHA-1 хеша, мы переходим в состояние «Detached HEAD», что полезно для просмотра истории коммитов.

Например, командой `git checkout v1.6.6.1` можно временно переключиться на тег, а затем вернуться в другую ветку (например, `git checkout main`).

Коммиты в состоянии «Detached HEAD» имеют свои особенности, которые будут рассмотрены далее



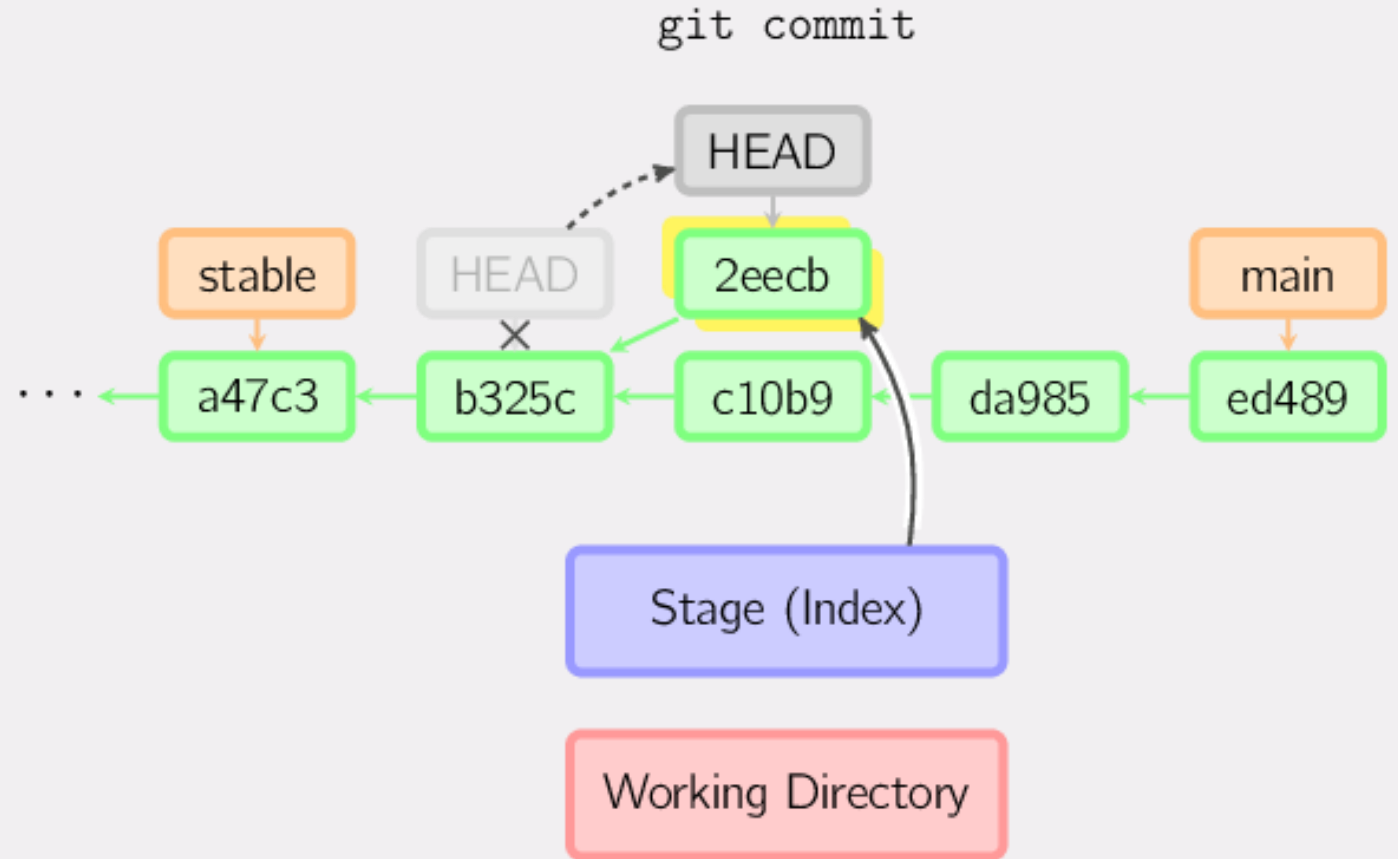
`git checkout main`

Команды Git

Коммит из состояния «Detached HEAD»

В состоянии «Detached HEAD» коммит выполняется как обычно, но ни один указатель ветки не изменяется.

Это похоже на работу с анонимной веткой.



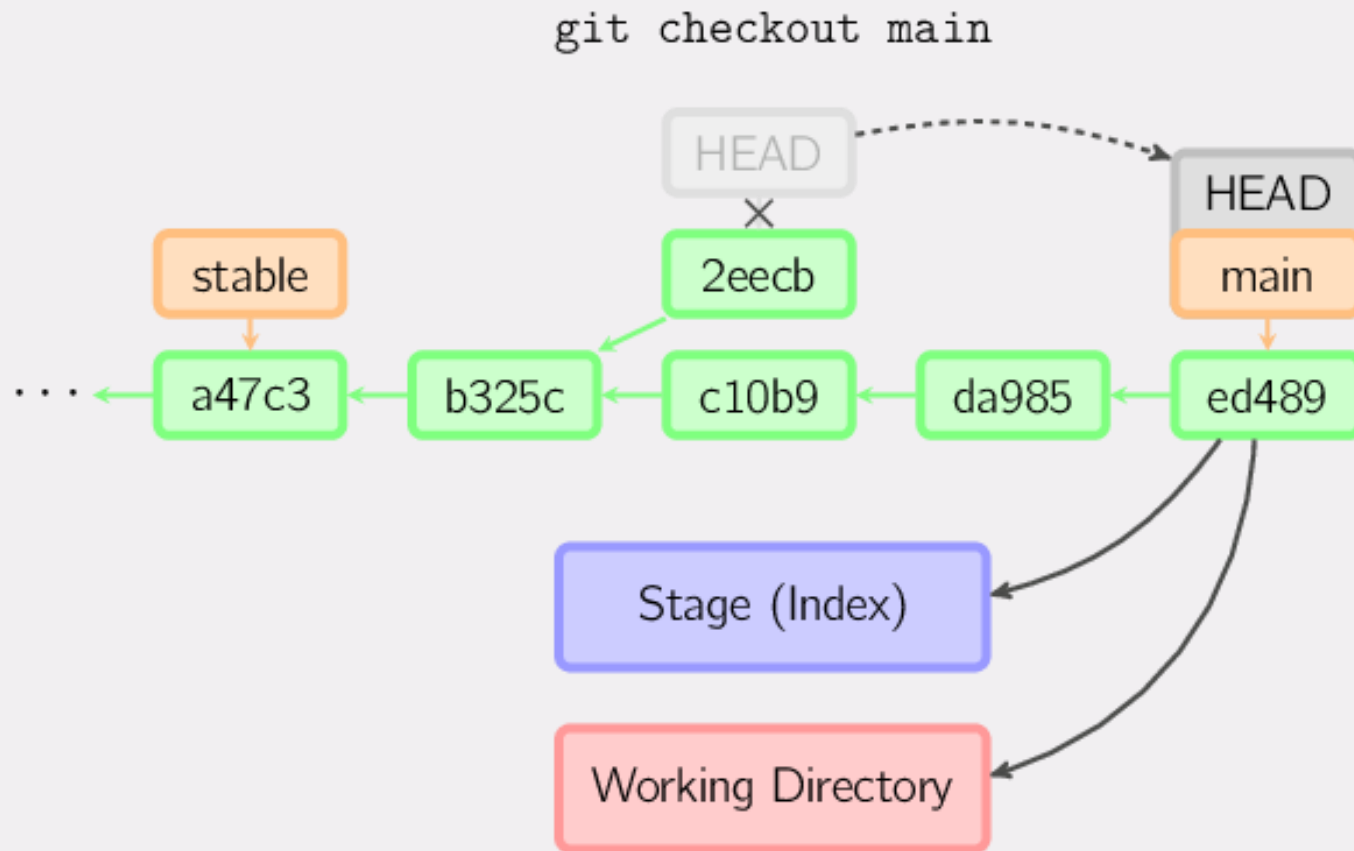
`git checkout main`

`git commit`

Команды Git

Коммит из состояния «Detached HEAD»

Если после такого коммита вы переключитесь на ветку `main`, коммит будет потерян и уничтожен сборкой мусора, так как на него нет ссылок.



`git checkout main`

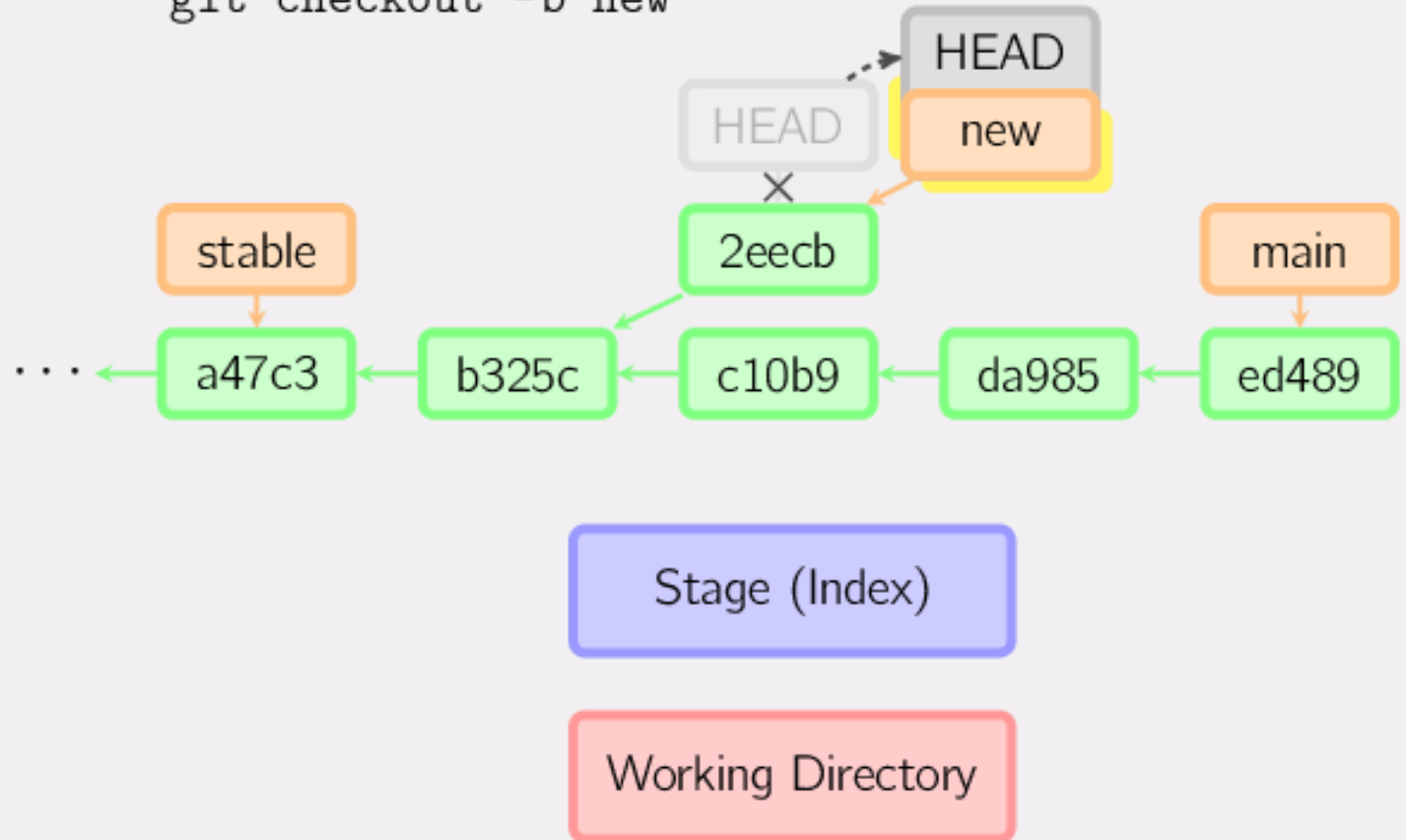
`git commit`

Команды Git

Чтобы сохранить такой коммит, создайте новую ветку с помощью команды

`git checkout -b new`

`git checkout -b new`



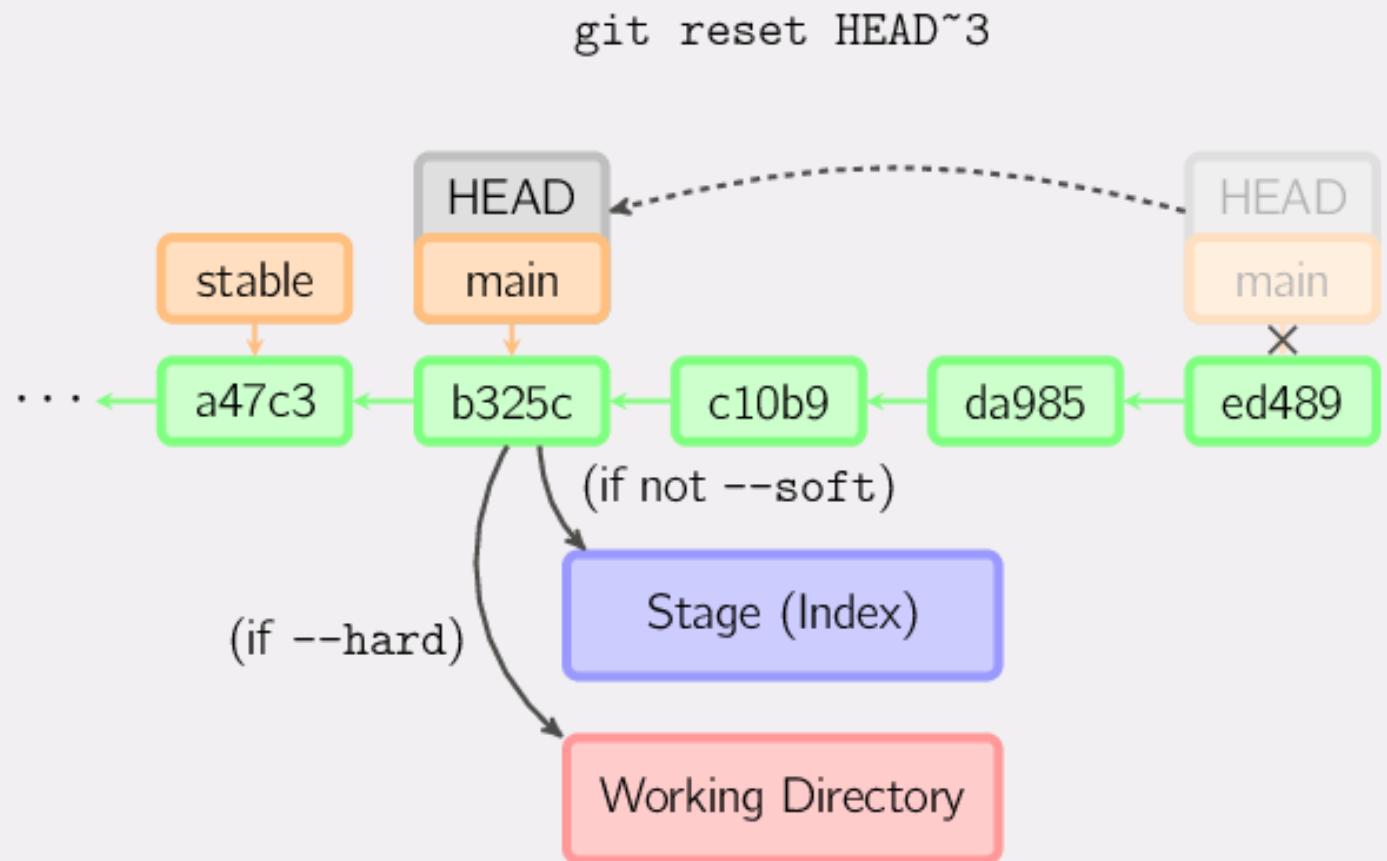
`git checkout -b new`

Команды Git

Команда **reset** перемещает указатель текущей ветки и может обновить сцену и рабочую директорию.

Если указать коммит без имен файлов, указатель переместится на него, а сцена обновится.

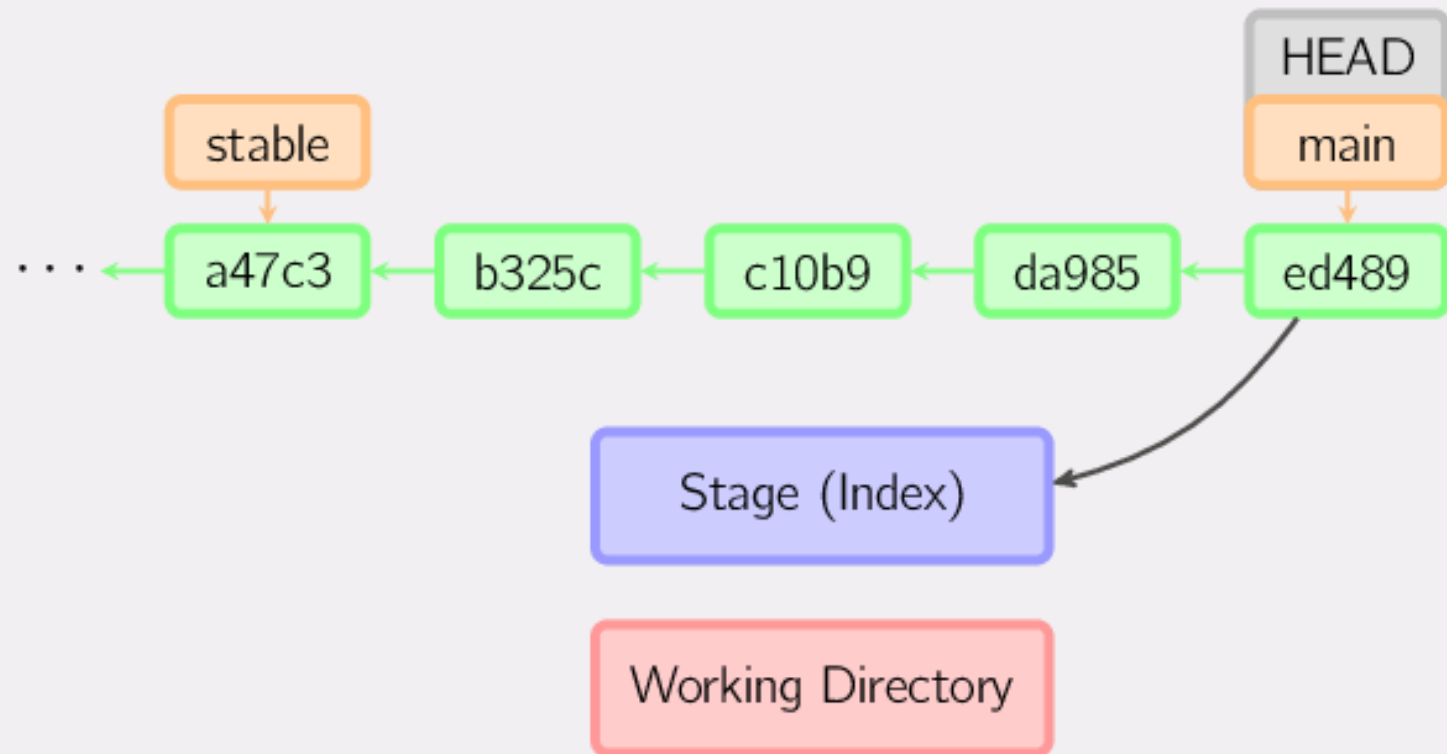
Ключи `--soft` и `--hard` определяют, будет ли изменена сцена или рабочая директория.



Команды Git

При указании имени файла команда работает аналогично checkout, но изменяет только сцену

```
git reset -- files
```



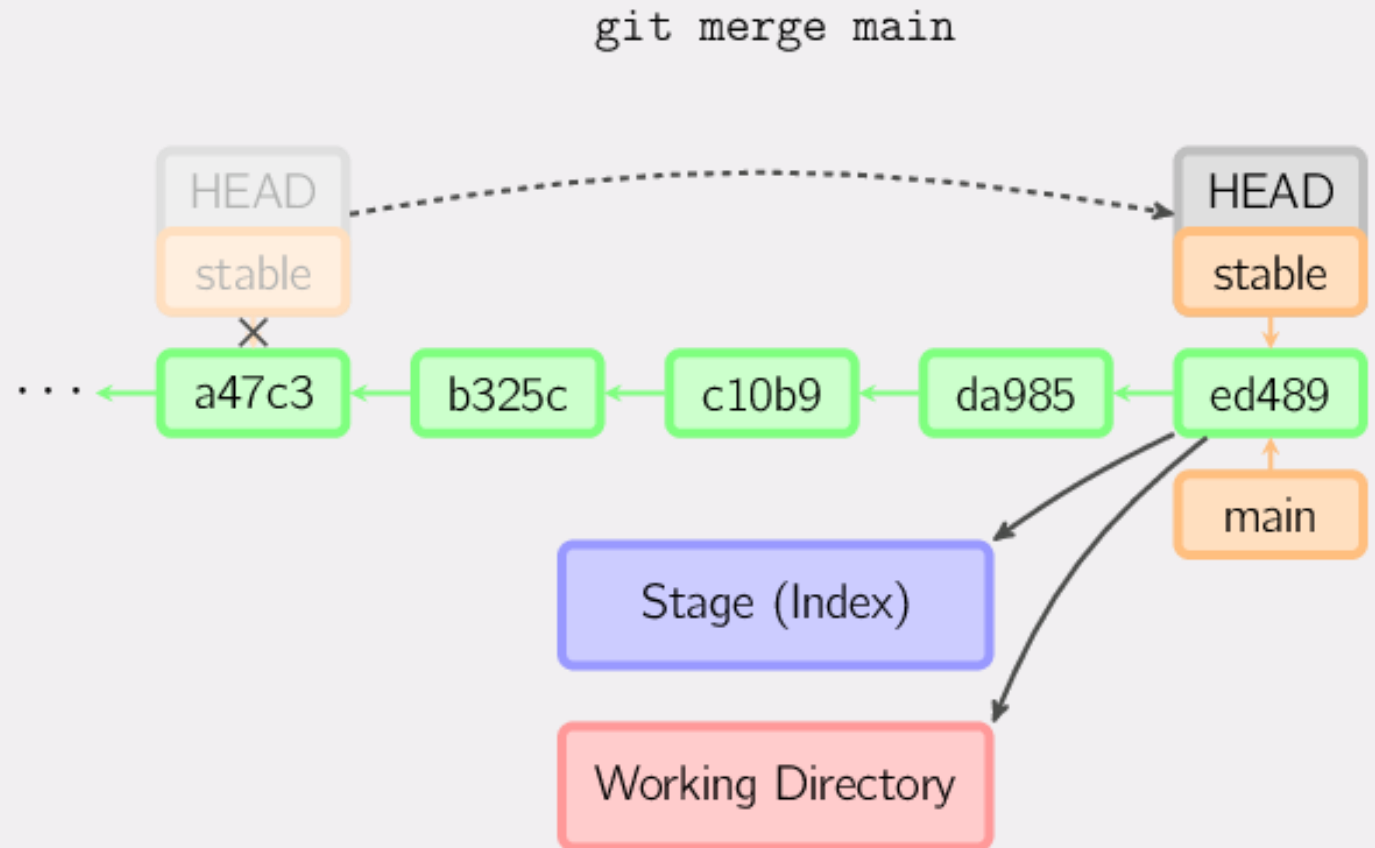
```
git reset --files
```

Команды Git

Команда **merge** создает новый коммит, применяя изменения других коммитов. Перед слиянием сцена должна соответствовать текущему коммиту.

Если другой коммит является предком текущего, ничего не происходит.

Если текущий — предок другого, выполняется быстрая перемотка (fast-forward), при которой ссылка текущей ветки перемещается на новый коммит.

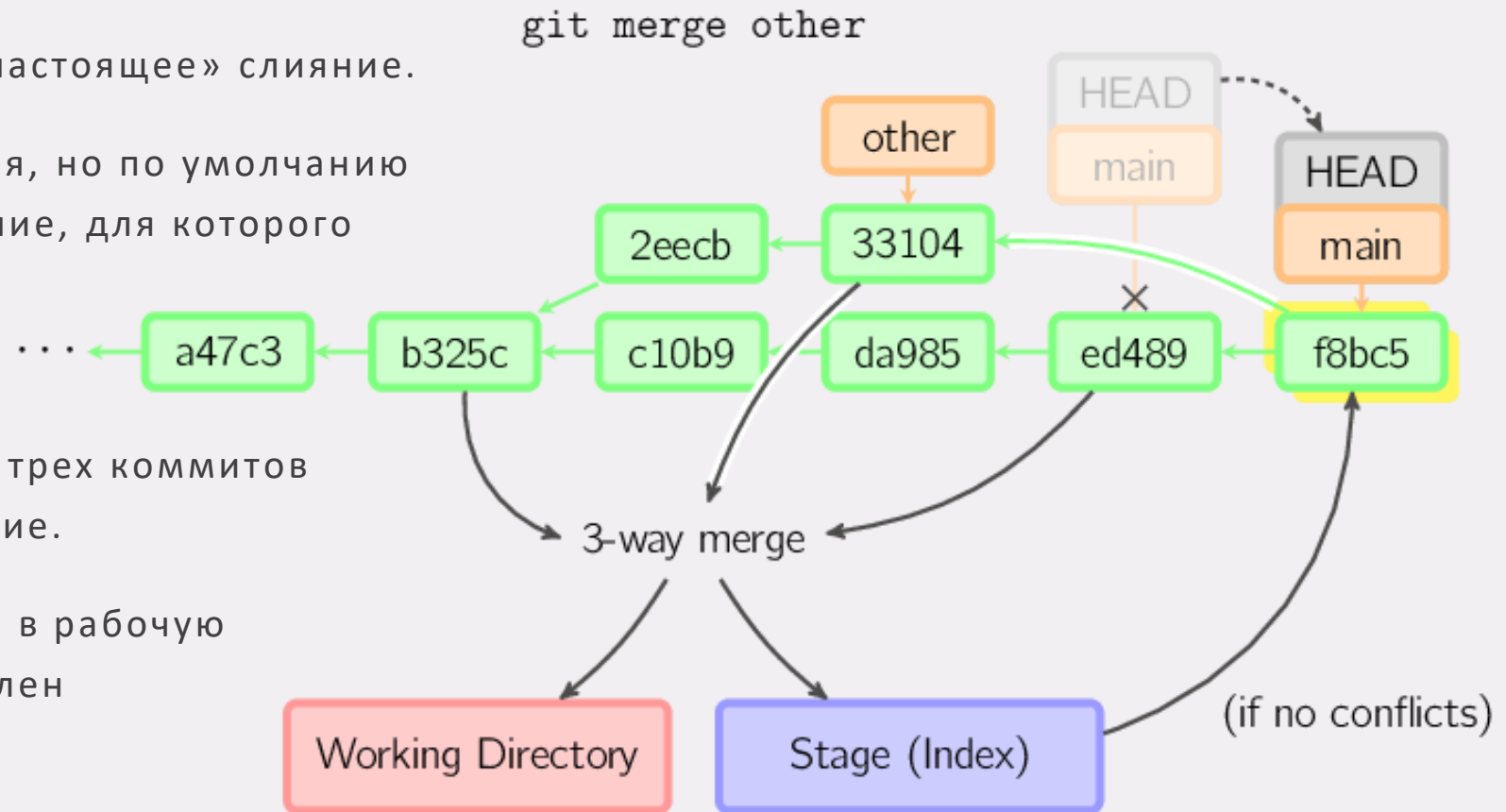


Команды Git

Во всех других случаях выполняется «настоящее» слияние.

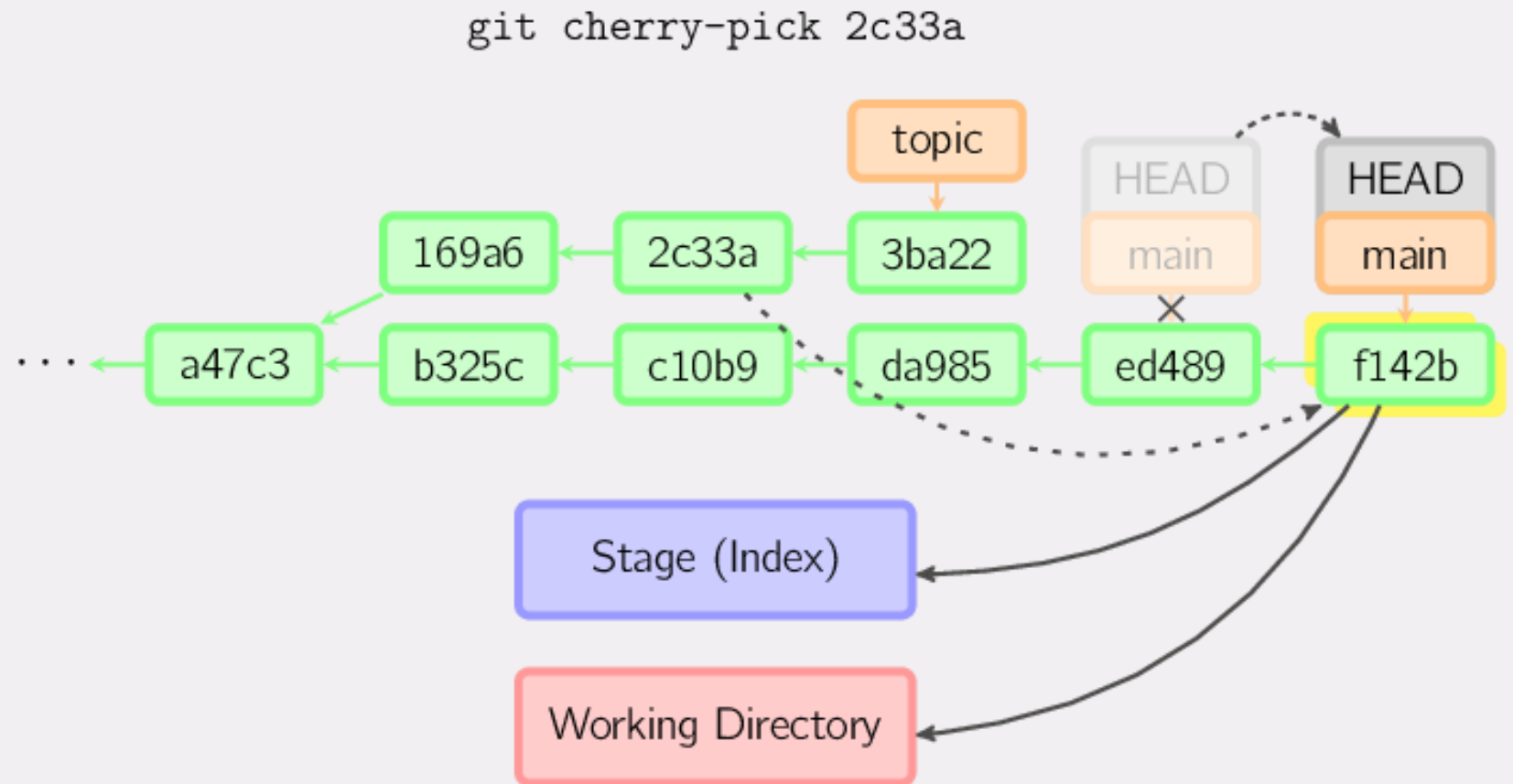
Вы можете изменить стратегию слияния, но по умолчанию будет выполнено «рекурсивное» слияние, для которого будет взят текущий коммит (ed489 ниже на схеме), другой коммит (33104) и их общий предок (b325c); и для этих трех коммитов будет выполнено трёхстороннее слияние.

Результат этого слияния будет записан в рабочую директорию и на сцену, и будет добавлен результирующий коммит со вторым родителем (33104)



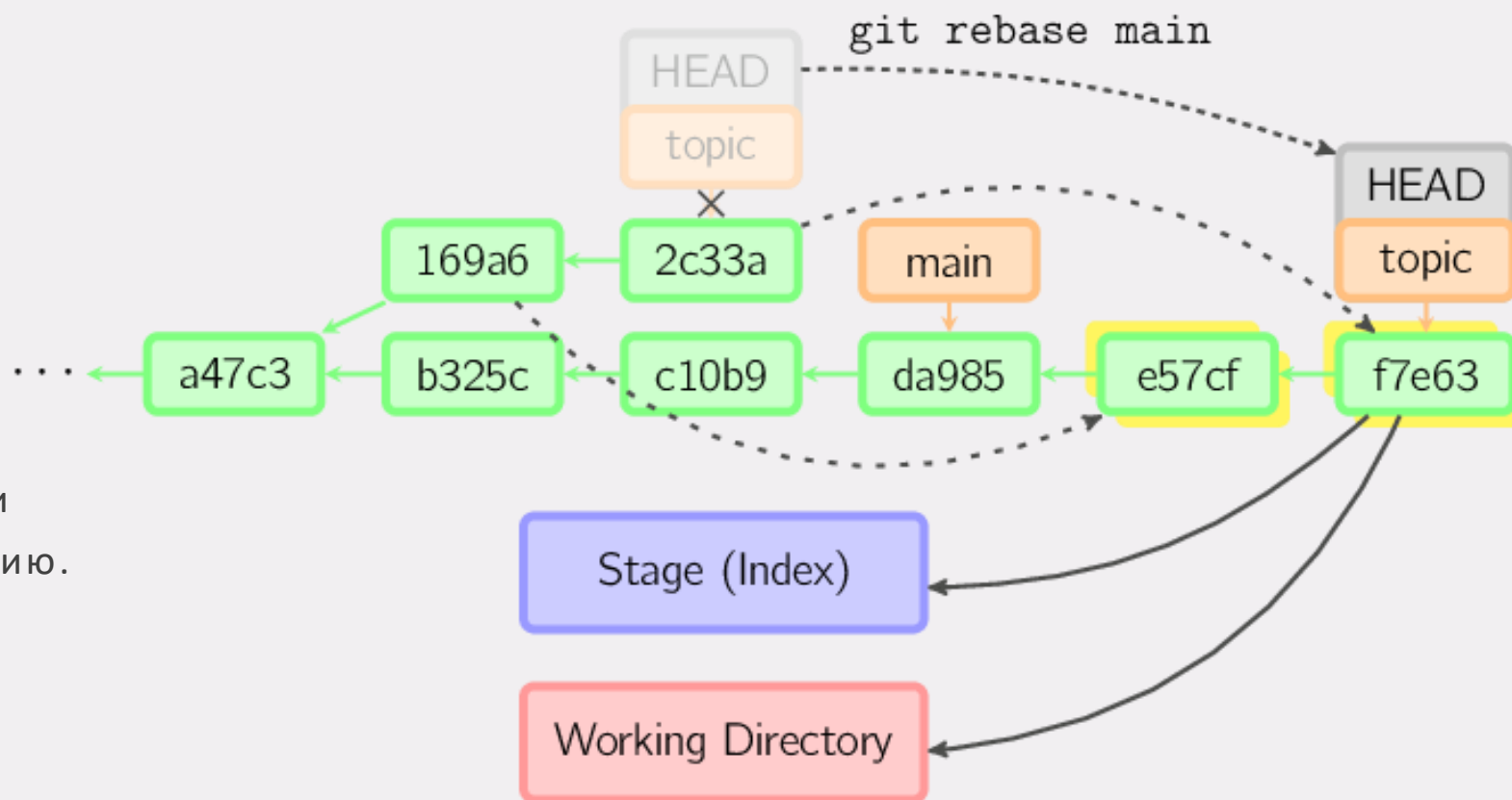
Команды Git

Команда **cherry-pick** создает новый коммит на основе одного выбранного коммита, применяя все его изменения и сообщение



Команды Git

Перебазирование (**rebase**) — альтернатива слиянию для объединения веток. В отличие от слияния, которое создает новый коммит с двумя родителями и нелинейной историей, перебазирование последовательно применяет коммиты из одной ветки в другую, оставляя линейную историю. Это фактически автоматическое выполнение нескольких команд **cherry-pick**.



На схеме видно, как команда берет коммиты из ветки `topic`, отсутствующие в ветке `main` (169a6 и 2c33a), и воспроизводит их в `main`.

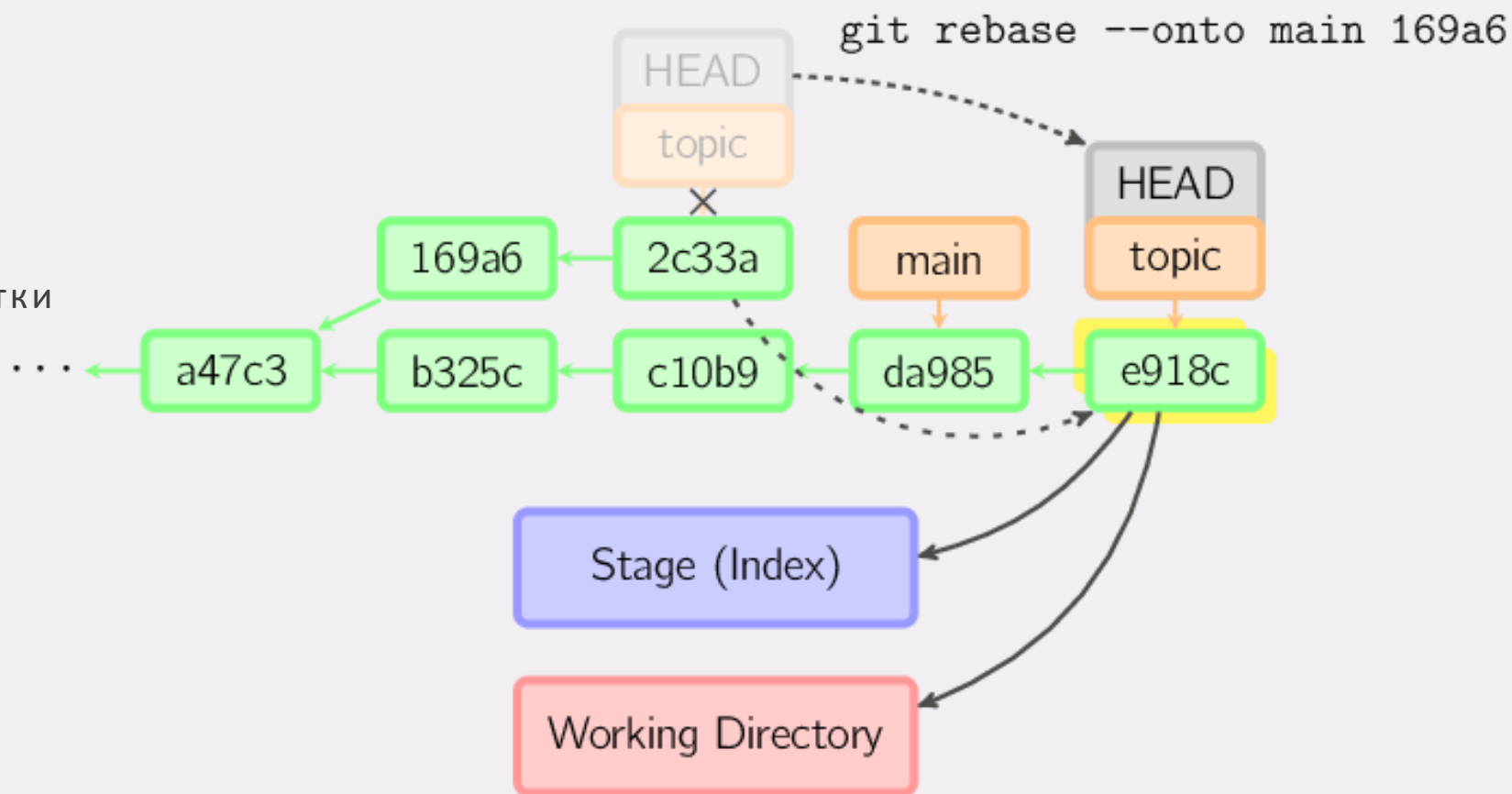
`git rebase main`

Указатель ветки перемещается на новое место.
Старые коммиты будут удалены, если на них нет ссылок

Команды Git

Используйте ключ `--onto`, чтобы ограничить глубину объединяемой ветки. На следующей схеме видно, что в ветку `main` приходят только последние коммиты из текущей ветки после (но не включая) `169a6`, т.е. `...` `a47c3` `b325c` `c10b9` `da985` `e918c`

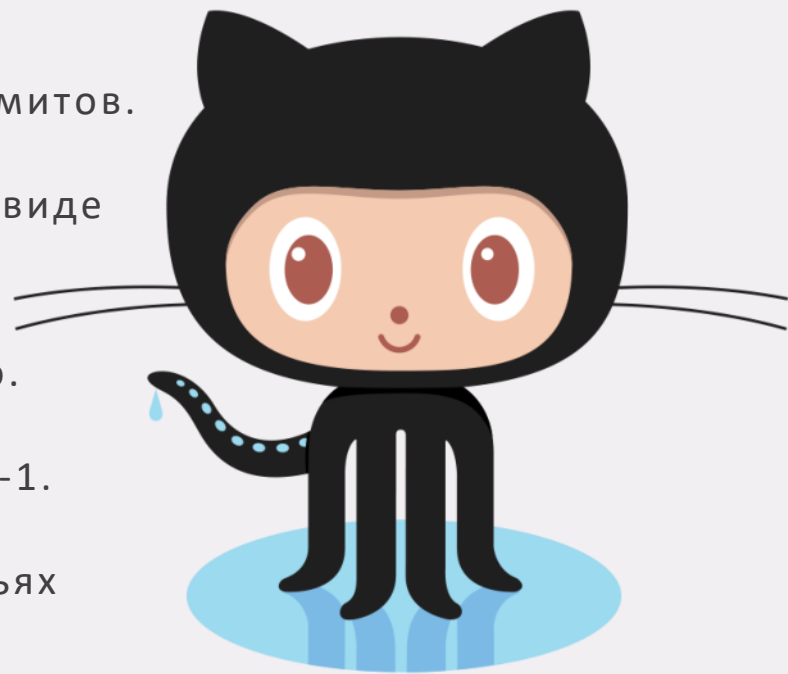
Существует интерактивный режим перебазирования `git rebase --interactive`, который позволяет изменять порядок, править и выдавливать коммиты



`git rebase --onto main 169a6`

Технические моменты связанные с работой с Git

- Содержимое файлов не хранится в индексе (`.git/index`) или объектах коммитов.
- Каждый файл хранится в базе данных объектов (`.git/objects`) в двоичном виде и идентифицируется по SHA-1 хешу.
- Индекс содержит имена файлов, их хеши и дополнительную информацию.
- Коммиты содержат тип данных дерево, также идентифицируемое по SHA-1.
- Деревья описывают директории и содержат информацию о других деревьях и файлах.
- При коммите из состояния «оторванной головы» (detached HEAD) на него ссылается история HEAD. Однако рано или поздно эта ссылка истечет, и коммит будет уничтожен сборщиком мусора, как при выполнении команд `git commit --amend` и `git rebase`.



Команды Git

Создание репозитория

Создать новый локальный репозиторий:
`$ git init {имя проекта}`

Скачать репозиторий:
`$ git clone {url}`

Состояние репозитория

Список незакоммиченных файлов:
`$ git status`

Изменения в ещё не добавленных в индекс файлах:
`$ git diff`

Изменения в уже добавленных:
`$ git diff --cached`

Все изменения:
`$ git diff HEAD`

Изменения между двумя коммитами:
`$ git diff hash1 hash2`

Получить даты изменений и их авторов:
`$ git blame {имя файла}`

Получить список изменений файла в определённом коммите:
`$ git show hash:file`

Список коммитов:
`$ git log`

Показать историю изменений, включая диффы:
`$ git log -p {файл или папка}`

Git

Работа с ветками

Список локальных веток:
`$ git branch`

Список всех веток:
`$ git branch -av`

Переключиться на ветку mybranch:
`$ git checkout mybranch`

Создать ветку mybranch и переключиться на неё:
`$ git checkout -b mybranch`

Удалить ветку mybranch:
`$ git branch -d mybranch`

Смержить ветку a в b:
`$ git checkout b && git merge a`

Создать тег для текущего коммита:
`$ git tag {имя тега}`

Сохраняем изменения

Добавить файл в индекс:
`$ git add {имя файла}`

Добавить все файлы в папке:
`$ git add .`

Закоммитить добавленные в индекс файлы с сообщением:
`$ git commit -m "сообщение"`

Закоммитить все файлы с сообщением:
`$ git commit -am "сообщение"`

Удалить изменённый файл из индекса, оставив изменения в нём:
`$ git reset {имя файла}`

Отменить все изменения в папке и вернуть всё к последнему коммиту:
`$ git reset --hard`

Обновление

Скачать изменения из удалённого репозитория:
`$ git fetch`

Скачать изменения из удалённого репозитория и смержить:
`$ git pull`

Скачать изменения из удалённого репозитория и наложить незапущенные коммиты поверх скачанных:
`$ git pull --rebase`

Отправить локальные коммиты в удалённый репозиторий:
`$ git push`

И главное!

Не забываем пользоваться подсказками:
`$ git {имя команды} --help`



GitHub Pages

GitHub Pages — часть платформы GitHub, заточенная специально под размещение сайтов в интернете



Изначально его придумали для рабочих нужд программистов, чтобы они могли публиковать свой код в интернете и смотреть, как он работает.

А потом его оценили и обычные пользователи. Так, на базе Pages часто делают сайты-визитки, портфолио, лендинги и другие проекты.

Устроено там всё просто: вы загружаете исходники сайта, а на выходе получаете ссылку на готовый ресурс в Сети.

Вам не нужно думать о домене, хостинге и безопасности. Сервис берёт все на себя.

GitHub Pages

- + Адаптитивность
- + Шаблоны оформления
- + Поддержка плагинов
- + Редактор кода



Сайты GitHub Pages могут быть доступны публично в интернете, даже если репозиторий является приватным.

Важно быть осторожным с конфиденциальными данными в репозиториях, так как они могут стать доступными после публикации.

Вы можете настроить автоматическую публикацию при каждом обновлении определенной ветки или написать рабочий процесс GitHub Actions для публикации сайта.



- Сайты GitHub Pages доступны в интернете, даже если репозиторий приватный (если это разрешено вашим планом).
- Изменения могут публиковаться до 10 минут после их загрузки на GitHub.
- GitHub Actions бесплатен для публичных репозиториев; за использование в частных и внутренних репозиториях могут взиматься сборы.

4

IDE + НАСТРОЙКИ + УСТАНОВКИ

IDE



Работать будем в Visual Studio Code.

Но, если удобно и есть понимание, можно использовать любое знакомое ПО/редактор кода: Atom, VS и другие

Visual Studio Code

IDE (Integrated Development Environment) — интегрированная среда разработки, комплекс программных средств, который используется для разработки программного обеспечения (ПО).

IDE объединяет все необходимые инструменты для работы в одном месте. В ней можно писать, проверять, тестировать и запускать код, не переключаясь между разными приложениями.

Visual Studio Code (VS Code) — текстовый редактор для разработчиков, разработанный компанией Microsoft. Предназначен для работы с различными языками программирования, а также для разработки кросс-платформенных десктопных приложений

Установка

Visual Studio Code



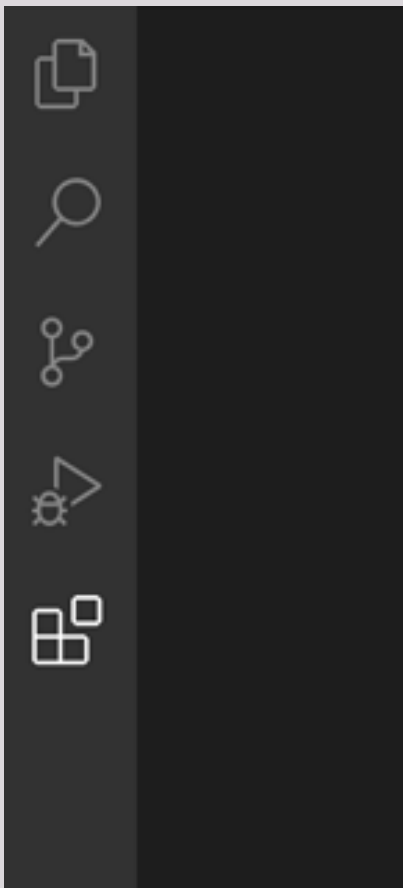
<https://code.visualstudio.com/>

Возможна установка на основные системы:

- [macOS](#)
- [Linux](#)
- [Windows](#)

Плагины

Visual Studio Code



Откройте раздел «Расширения», нажав на значок «Расширения» на панели активности, или воспользуйтесь сочетанием клавиш **Ctrl+Shift+X**.

При желании вы можете выполнить поиск расширения с помощью поля поиска или одного из фильтров. Например, чтобы отфильтровать расширения по категориям.

Выберите расширение, чтобы просмотреть его сведения, такие как описание, информация об издателе, количество установок, рейтинг пользователей и многое другое.

Установка



<https://git-scm.com/install/>



GIT

Проверяем есть ли гит

Необходимо открыть консоль (или терминал) и выполнить команду. Данная проверка подходит для всех операционных систем

```
git --version
```


Установка

GIT

Для Ubuntu или Debian выполните:

```
sudo apt update  
sudo apt install git
```

Для Fedora:

```
sudo dnf install git
```

Для Arch Linux:

```
sudo pacman -S git
```

Установка на Linux

В зависимости от вашего дистрибутива Linux требуется установить через консоль, например в убунту эта команда будет иметь вид:

```
sudo apt install git
```

Установка на macOS

Установщик Homebrew проведёт через все необходимые шаги.

```
brew install git
```





<https://git-scm.com/install/>



Установка в Windows

Скачайте exe-файл инсталлятора с сайта и запустите его.

Это Git для Windows, он называется msysGit. Установщик спросит добавлять ли в меню проводника возможность запуска файлов с помощью Git Bash (консольная версия) и GUI (графическая версия). Подтвердите действие, чтобы далее вести работу через консоль в Git Bash. Остальные пункты можно оставить по умолчанию.

Настройка

По умолчанию консоль VS Code в Windows запускает стандартную командную строку Windows, которая не может работать с GIT, чтобы все работало нужно запустить терминал bash, для этого выполним следующие действия:

GIT + VS

1. Убедимся, что ранее нами был скачан и установлен git.
2. Запускаем Visual Studio Code.
3. Жажимаем сочетание клавиш Ctrl + Shift + P вводим open user setting и из выпадающего меню выбираем Open User Settings.
4. В параметры поиска вводим terminal.integrated.shell.windows и заменяем cmd.exe на C:\\Program Files\\Git\\bin\\bash.exe (если вы установили git в другую директорию то ссылка будет иметь вид относительно установленной вами директории).
5. Перезагружаем Visual Studio Code
6. Открываем терминал и проверяем, что запущен bash терминал



Также могут возникнуть следующие трудности:

1. Запрет запуска скриптов в системе. Решается путем выполнения в PowerShell команды

Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

(разрешает запуск скриптов PowerShell для текущего пользователя).
По умолчанию в Windows запуск скриптов блокируется

2. Настройки переменных сред.

Решение можно посмотреть по ссылке:



<https://www.delftstack.com/howto/git/add-git-to-path-on-windows/>

Настройка

Обратите внимание, что в командах, есть опция **--global**.

Это значит, что такие данные будут сохранены для всех ваших действий в Git и вводить их больше не надо.

Если вы хотите менять эту информацию для разных проектов, то в директории проекта вводите эти же команды, только без опции **--global**

GIT + VS

После установки производим настройку своего профиля вводя в терминал поочередно команды:

```
git config --global user.name "ВАШЕ_ИМЯ"
```

```
git config --global user.email АДРЕС
```

Заменяв значения ВАШЕ_ИМЯ и АДРЕС вашими значениями. После указания своих данных, можно их просмотреть:

```
git config --global --list
```

Вспомогательные файлы проекта



README.md — это визитная карточка вашего проекта. Файл создается в формате Markdown (.md) и должен содержать исчерпывающую информацию для тех, кто впервые видит ваш проект: название, описание, технологии, инструкции по установке и запуску, примеры использования. Наличие качественного README — признак профессионализма.

.gitignore — это конфигурационный файл, в котором указываются шаблоны файлов и папок, которые Git должен игнорировать и не добавлять в репозиторий. Это необходимо для того, чтобы не засорять историю проекта служебными файлами операционной системы, редактора кода, зависимостями (например, папка `node_modules`) или файлами с чувствительными данными (пароли, ключи).

**СПАСИБО ЗА
ВНИМАНИЕ!**

12 ноября 2025 года